

VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

(An Autonomous Institute Affiliated to University of Mumbai)
Department of Computer Engineering



Project Report on

Civility AI: Foul Language Detection System

In partial fulfilment of the Fourth Year, Bachelor of Engineering (B.E.) Degree in
Computer Engineering at the University of Mumbai
Academic Year 2025 - 26

By

Rahul Dudani - D17C/18
Aanchal Galani - D17C/20
Sumeet Janyani - D17C/32
Ved Shirur - D17C/60

Project Mentor

Dr Prashant Kanade

(A.Y. 2025 - 26)

VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

(An Autonomous Institute Affiliated to University of Mumbai)

Department of Computer Engineering



Certificate

This is to certify that **Rahul Dudani (D17C - 18), Aanchal Galani (D17C - 20), Sumeet Janyani (D17C - 32) & Ved Shirur (D17C - 60)** of Fourth Year Computer Engineering studying under the University of Mumbai have satisfactorily completed the project on **“Foul Language Detection System”** as a part of their coursework of PROJECT - II for Semester - VIII under the guidance of their mentor **Dr Prashant Kanade** in the year 2025 - 26.

This thesis/dissertation/project report entitled **“Civility AI: Foul Language Detection System”** by **Rahul Dudani, Aanchal Galani, Sumeet Janyani & Ved Shirur** is approved for the degree of **Bachelor of Engineering in Computer Engineering**.

Programme Outcomes	Grade
PO1, PO2, PO3, PO4, PO5, PO6, PO7, PO8, PO9, PO10, PO11, PO12 PSO1 & PSO2	

Date: 27th April, 2025

Project Guide: **Dr Prashant Kanade**

Project Report Approval

For

B. E (Computer Engineering)

This project report entitled “Civility AI: Foul Language Detection System” by **Rahul Dudani (D17C - 18), Aanchal Galani (D17C - 20), Sumeet Janyani (D17C - 32) & Ved Shirur (D17C - 60)** is approved for the degree of **Bachelor of Engineering in Computer Engineering**.

Examiners

1.

(Internal Examiner name & sign)

2.

(External Examiner name & sign)

3.

(Head of Department)

4.

(Principal)

Date: 27th April, 2025

Place: VESIT, Mumbai

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

.....
Rahul Dudani (D17C - 18)

.....
Aanchal Galani (D17C - 20)

.....
Sumeet Janyani (D17C - 32)

.....
Ved Shirur (D17C - 60)

Date :

Acknowledgement

We are thankful to our college Vivekanand Education Society's Institute of Technology for considering our project and extending help at all stages needed during our work of collecting information regarding the project.

It gives us immense pleasure to express our deep and sincere gratitude to the Head of the Computer Department **Dr. (Mrs.) Nupur Giri** (Project Guide) for her kind help and valuable advice during the development of project synopsis and for her guidance and suggestions.

We are deeply indebted to our Principal **Dr. (Mrs.) J.M. Nair**, for giving us this valuable opportunity to do this project.

We express our hearty thanks to them for their assistance without which it would have been difficult to finish this project synopsis and project review successfully.

We convey our deep sense of gratitude to all teaching and non-teaching staff for their constant encouragement, support and selfless help throughout the project work. It is a great pleasure to acknowledge the help and suggestion, which we received from the Department of Computer Engineering.

We wish to express our profound thanks to all those who helped us in gathering information about the project. Our families too have provided moral support and encouragement several times.

COURSE OUTCOMES FOR B.E. PROJECT

Learners will be able to,

Course Outcome	Description of the Course Outcome
CO1	Able to apply the relevant engineering concepts, knowledge and skills towards the project.
CO2	Able to identify, formulate and interpret the various relevant research papers and to determine the problem.
CO3	Able to apply the engineering concepts towards designing solutions for the problem.
CO4	Able to interpret the data and datasets to be utilized.
CO5	Able to create, select and apply appropriate technologies, techniques, resources and tools for the project.
CO6	Able to apply ethical, professional policies and principles towards societal, environmental, safety and cultural benefit.
CO7	Able to function effectively as an individual, and as a member of a team, allocating roles with clear lines of responsibility and accountability.
CO8	Able to write effective reports, design documents and make effective presentations.
CO9	Able to apply engineering and management principles to the project as a team member.
CO10	Able to apply the project domain knowledge to sharpen one's competency.
CO11	Able to develop a professional, presentational, balanced and structured approach towards project development.
CO12	Able to adopt skills, languages, environment and platforms for creating innovative solutions for the project.

Index

Chapter No.	Title	Page No.
1	Introduction 1.1. Introduction to the project 1.2. Motivation for the project 1.3. Problem Definition 1.4. Relevance of the Project 1.5. Methodology used	2
2.	Literature Survey 2.1. Research Papers a. Abstract of the research paper b. Inference drawn from the paper 2.2. Books / Articles referred / news paper referred 2.3. Interaction with domain experts. 2.4. Patent search you need for study 3-4 patents related to your project, a. Title of the patent and year of the patent b. Summary of the patent c. Link 1. European Patent: http://worldwide.espacenet.com/, Link 2. USpatent http://patft.uspto.gov/netahtml/PTO/index.html	6
3.	Requirement Of Proposed System 3.1 Functional Requirements 3.2. Non-Functional Requirements 3.3. Constraints 3.4. Hardware & Software Requirements 3.5. Techniques utilized till date for the proposed system 3.6. Constraints of Working	15
4.	Proposed Design 4.1 Block diagram representation of the proposed system Explanation for the block diagram 4.2. Modular diagram representation of the proposed system Explanation for the modular block diagram 4.3 Design of the proposed system with proper explanation of each : a. Data Flow Diagrams e. Screen shot of implementation 4.4 . Project Scheduling & Tracking using Timeline / Gnatt Chart	21

5.	Proposed Results and Discussions 5.1. Methodology employed for development 5.2 Algorithms and flowcharts for the respective modules developed 5.3 Datasets source and utilization	24
6.	Testing of the Proposed System 6.1 . Introduction to testing 6.2. Types of tests Considered 6.3 Various test case scenarios considered 6.4. Inference drawn from the test cases	29
7.	Results and Discussion 7.1. Screenshots of User Interface (UI) for the respective module 7.2. Performance Evaluation measures 7.3. Input Parameters / Features considered 7.4. Graphical and statistical output 7.5. Comparison of results with existing systems 7.6. Inference drawn	34
8.	Conclusion 8.1 Limitations 8.2 Conclusion 8.3 Future Scope	45
9.	References (In IEEE format)	47
10.	Appendix 9.1.List Of Figures 9.2.List Of Tables 9.3.Paper Publications a. Draft of the paper / Paper Published b. Xerox of project review sheet	49

a. List of Figures

Figure Number	Heading	Page no.
4.1	Block Diagram	21
4.2	Modular Diagram	22
4.3	DFD Level 0	22
5.1	Flowchart System Flow	27
7.1	Civility AI Landing Page Interface	34

7.2	Text File Refinement	35
7.3	Real-Time Civility Check and Input Restriction	36
7.4	Image Refinement	37
7.5	Accuracy Comparison Across Modules	41
7.6	Civility Score Distribution	42
7.8	Performance Metrics Comparison	42

b. List of tables

Table Number	Heading	Page no.
Table 2.1	Comparison of Research Papers	8
Table 2.2	Lacuna of existing system and proposed work	15
Table 7.5	Comparison of Civility AI with Existing Systems	37
Table 6.3	Comprehensive Test Cases for Civility AI System	53

ABSTRACT

The rapid expansion of digital communication platforms has led to a significant increase in user-generated content, which often includes offensive, abusive, and foul language. This growing issue negatively impacts online environments by promoting toxicity, cyberbullying, and hate speech. To address this challenge, the proposed system, *Civility AI: Foul Language Detection System*, utilizes Natural Language Processing (NLP) and Machine Learning (ML) techniques to automatically detect and classify offensive content in real time.

The system is designed to overcome the limitations of traditional rule-based approaches by incorporating advanced text representation methods such as TF-IDF, Word2Vec, and GloVe, enabling better contextual understanding of language. It supports multilingual and code-mixed text, making it suitable for diverse linguistic environments commonly found on social media and online platforms. Various machine learning models, including Logistic Regression, Naive Bayes, and Support Vector Machines, are employed and evaluated to ensure optimal performance.

Additionally, the system focuses on minimizing false positives while maintaining high accuracy and adaptability to evolving language patterns, including slang and disguised offensive terms. A modular architecture allows easy scalability and integration across platforms such as social media, gaming chats, and educational systems. The solution also incorporates evaluation metrics like precision, recall, and F1-score to ensure reliability and effectiveness.

Overall, Civility AI aims to promote healthier online interactions by providing an efficient, scalable, and context-aware solution for detecting and mitigating foul language in digital communication.

Chapter 1: Introduction

1.1 Introduction :

The rapid growth of the internet and the widespread adoption of social media platforms have transformed the way people communicate and share information. Platforms such as messaging apps, forums, and social networks allow users to express their thoughts, opinions, and emotions instantly to a global audience. While this digital revolution has enhanced connectivity and freedom of expression, it has also resulted in the uncontrolled spread of offensive, abusive, and inappropriate language online.

Toxic content such as hate speech, cyberbullying, and profanity not only degrades the quality of online discussions but also causes serious psychological harm to individuals. It can lead to anxiety, depression, and social isolation, especially among young users. Moreover, such behavior can escalate into real-world conflicts and negatively impact societal harmony.

Traditional moderation techniques, including manual review and keyword-based filtering, are insufficient to handle the massive volume of content generated every second. Manual moderation is time-consuming, inconsistent, and expensive, while keyword-based systems fail to capture context and are easily bypassed.

To overcome these challenges, *Civility AI* is proposed as an intelligent foul language detection system. It leverages Natural Language Processing (NLP) and Machine Learning (ML) techniques to automatically detect and classify offensive content in real-time. The system performs text preprocessing, feature extraction using TF-IDF and word embeddings such as Word2Vec and GloVe, and applies machine learning models like Logistic Regression, Naive Bayes, and Support Vector Machines (SVM). It is designed to work effectively across multilingual and code-mixed data, ensuring accurate and context-aware moderation for a safer digital environment.

1.2 Motivation

In today's digital era, the volume of user-generated content is growing exponentially across various platforms. Social media platforms, online forums, and gaming communities often face large-scale misuse in the form of abusive language, hate speech, trolling, and harassment. This not only affects individual users but also damages the credibility and usability of these platforms.

Existing moderation approaches, such as keyword filtering and manual review, have significant limitations. Keyword-based systems lack contextual understanding and often misinterpret harmless content as offensive, leading to high false positives. On the other hand, users frequently use slang, abbreviations, creative spellings, or emojis to bypass such filters. Manual moderation, while accurate, is not scalable and cannot keep up with real-time content generation.

The motivation behind developing *Civility AI* is to create a robust, scalable, and context-aware system that can intelligently identify offensive language while minimizing errors. The system aims to adapt to evolving language patterns, including new slang and cultural variations, making it suitable for diverse user bases.

Additionally, the project is driven by the need to create safer online spaces where users can interact without fear of harassment or abuse. By promoting respectful communication, *Civility AI* contributes to improving the overall quality of digital interactions across social media, educational platforms, and professional environments.

1.3 Problem Definition

The primary challenge addressed in this project is the accurate and efficient detection of offensive language in online content across diverse linguistic and contextual scenarios. Unlike structured data, natural language is highly complex, ambiguous, and constantly evolving, making it difficult to classify content based solely on predefined rules. Users often attempt to bypass moderation systems by using creative techniques such as misspellings, special characters, emojis, abbreviations, or mixing multiple languages within a single sentence (code-mixing). These variations make it difficult for traditional systems to identify offensive content accurately. Furthermore, the same word may have different meanings depending on the context, making it

challenging to distinguish between harmful and harmless usage. Conventional blacklist-based approaches are limited in their ability to understand semantics and context, leading to incorrect classifications. These systems often generate high false positives (flagging non-offensive content) and false negatives (missing actual offensive content). To address these challenges, this project focuses on developing a machine learning-based foul language detection system with the following capabilities:

- Handling multilingual and code-mixed text effectively
- Understanding contextual meaning rather than relying only on keyword matching
- Minimizing false positives and false negatives for reliable moderation
- Adapting to evolving slang, trends, and new forms of online expression
- Providing scalable and real-time detection suitable for high-volume platforms

1.4 Relevance of the Project

The Civility AI system holds significant importance in today's digital landscape, where maintaining a safe and respectful online environment is becoming increasingly critical. With the rise in cyberbullying, hate speech, and toxic interactions, there is a growing need for automated solutions that can assist in effective content moderation.

This project is highly relevant in the fields of digital governance, social media management, and educational technology. By automating the detection of offensive language, it reduces the burden on human moderators, improves efficiency, and ensures consistent decision-making.

The modular and scalable design of Civility AI allows it to be integrated into various domains, including:

- **Social Media Platforms:** Automatically filtering or flagging abusive comments and posts
- **Gaming and Chat Applications:** Promoting respectful communication among players
- **Corporate Communication Systems:** Maintaining professionalism in workplace interactions
- **Educational Platforms:** Creating safe learning environments for students

Furthermore, the system can be extended with additional features such as sentiment analysis, toxicity scoring, and real-time alerts, enhancing its practical usability. By fostering positive communication and reducing harmful content, Civility AI plays a vital role in building a healthier and more inclusive digital ecosystem.

1.5 Methodology Used

The Civility AI system follows a structured methodology comprising six main stages:

1. Data Collection and Preprocessing:

- Datasets used include *Hate Speech and Offensive Language Dataset*, *OLID (Offensive Language Identification Dataset)*, and *FIRE 2021 Dravidian CodeMix*.
- Steps: noise removal, tokenization, stopwords elimination, text normalization, and dataset balancing.

2. Feature Extraction:

- TF-IDF used for frequency-based representation.
- GloVe and Word2Vec embeddings used for capturing semantic context.
- A custom lexicon of offensive words integrated for enhanced detection.

3. Model Development:

- Algorithms: Logistic Regression, Naive Bayes, and SVM.
- Comparative evaluation based on precision, recall, F1-score, and ROC-AUC.

4. Multilingual Handling:

- Language detection followed by embedding-specific preprocessing for regional or code-mixed texts.

5. Evaluation and Optimization:

- Cross-validation using metrics like Accuracy, Kappa Score, and Matthews Correlation Coefficient.

6. Deployment and Feedback:

- The trained model deployed via Flask/FastAPI.
- Optional moderator interface for semi-supervised improvement through user feedback.

Chapter 2: Literature Survey

2.1 Research Papers Referred

[1] Davidson, T. et al. (2017) - Automated Hate Speech Detection and the Problem of Offensive Language

Abstract (in own words): The paper highlights how machine learning can distinguish hate speech from offensive but non-hateful content. It uses logistic regression with TF-IDF features to classify tweets.

Inference: Inspired Civility AI's feature extraction and ML classifier comparison approach.

[2] Waseem, Z., & Hovy, D. (2016) - Predictive Features for Hate Speech Detection on Twitter

Abstract: The authors examine linguistic and contextual cues for hate speech detection using annotated datasets.

Inference: Helped identify key linguistic patterns and the need for context-awareness in Civility AI.

[3] Founta, A. M. et al. (2018) - Large Scale Crowdsourcing and Characterization of Twitter Abusive Behavior

Abstract: Introduced a large annotated dataset for abusive tweet classification and compared multiple ML models.

Inference: Provided insights on dataset design and annotation balance strategies.

[4] Mishra, P. et al. (2019) - Abusive Language Detection with Graph Convolutional Networks

Abstract: Proposed a deep learning approach for understanding text relationships using graph networks.

Inference: Highlighted advanced alternatives like GCNs, guiding future extensions of Civility AI.

Paper / Source	Special Features	Limitations
T. Davidson, D. Warmley, M. Macy, I. Weber, “Automated Hate Speech Detection and the Problem of Offensive Language,” ICWSM, 2017.	● Introduces a machine learning approach using Logistic Regression and TF-IDF for detecting offensive and hate speech on Twitter. ● Presents a large, annotated dataset distinguishing <i>hate speech</i> from <i>offensive but non-hateful</i> content.	● Dataset limited to English-language tweets. ● Lacks contextual and cultural understanding, often misclassifying sarcasm or coded words.
Z. Waseem, D. Hovy, “Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter,” NAACL-HLT, 2016.	● Uses linguistic and contextual cues to distinguish between different categories of hate speech. ● Highlights the importance of balanced annotation and inclusion of minority viewpoints.	● Focuses on small, manually labeled datasets. ● Limited generalization to multilingual and informal contexts. ● Relies heavily on handcrafted features, reducing scalability.
A. M. Founta, C. Djouvas, D. Chatzakou, et al., “Large Scale Crowdsourcing and Characterization of Twitter Abusive Behavior,” ICWSM, 2018.	● Introduces one of the largest crowdsourced datasets for offensive and abusive behavior detection. ● Classifies data into multiple categories—abusive, hateful, spam, and normal.	● Labeling inconsistency due to subjective human judgment. ● Dataset focused mainly on Western linguistic patterns. ● Does not fully address context-based offensive expressions.
P. Mishra, H. Yannakoudakis, E. Shutova, “Abusive Language Detection with Graph Convolutional Networks,” ACL, 2019.	● Applies Graph Convolutional Networks (GCNs) to capture syntactic and contextual dependencies. ● Outperforms traditional models by learning relationships between words dynamically. ● Demonstrates improved interpretability through graph-based representation.	● Computationally expensive and requires large GPU resources. ● Not optimized for real-time applications. ● Limited explainability in multilingual scenarios.

R. Sharma, D. Patel, A. Kumar, “Offensive Language Detection on Social Media using Machine Learning,” IJIRCCE, 2020.	● Compares Logistic Regression, SVM, and Naive Bayes for text classification. ● Combines TF-IDF and Word2Vec for hybrid feature extraction. ● Achieves high precision on benchmark English datasets.	● Model performance drops on code-mixed or non-English datasets. ● Does not utilize deep learning or contextual embeddings. ● Evaluation limited to static datasets.
S. Banerjee, P. Roy, “Toxic Comment Classification using Deep Learning Techniques,” IEEE Access, 2021.	● Employs CNN and BiLSTM for contextual text classification. ● Captures sequential dependencies and sentiment variations in toxic comments. ● Reduces false positives through better semantic understanding.	● Requires large labeled data and higher training time. ● Difficult to interpret due to black-box deep learning nature. ● Focused primarily on English-language data.
A. Gupta, K. Thomas, “A Multilingual Approach for Offensive Language Detection in Code-Mixed Data,” Springer LNCS, 2022.	● Focuses on multilingual and code-mixed text (English–Hindi) for offensive content detection. ● Integrates word embeddings with transformer-based architecture for better cross-language understanding. ● Addresses region-specific slang and transliteration challenges.	● Transformer models require high computational power. ● Limited availability of labeled code-mixed datasets. ● Still struggles with sarcasm and implicit abuse.

Table 2.1 : Comparison of Research Papers

2.2.Books/ Journals/ Articles referred

1. Foundations of Statistical Natural Language Processing – Christopher Manning & Hinrich Schütze

This book provides a strong foundation in the statistical approaches used in Natural Language Processing (NLP). It discusses essential techniques like tokenization, stemming, lemmatization, and probabilistic modeling of language. The text also elaborates on the mathematical concepts underlying text classification, information retrieval, and machine translation.

Inference Drawn: The book helped in understanding the mathematical and probabilistic base of NLP, enabling the development of robust text preprocessing and feature extraction pipelines in Civility AI.

2. **Speech and Language Processing – Daniel Jurafsky & James H. Martin**

This comprehensive book explains the theoretical and practical aspects of computational linguistics and NLP. It covers syntactic parsing, word embeddings, semantic analysis, and sequence modeling techniques like Hidden Markov Models and neural networks.

Inference Drawn:

The reference was useful in understanding the role of contextual semantics and how linguistic models can be adapted for offensive language detection in multilingual settings.

3. **Ethics and Artificial Intelligence in Social Media Moderation – Journal of Digital Society Studies, Vol. 14, 2021**

This journal article explores ethical concerns and biases associated with automated moderation systems. It discusses the impact of algorithmic decisions on free speech and fairness in AI-driven content filtering.

Inference Drawn:

The article influenced Civility AI's focus on transparency and explainability, motivating the integration of interpretability tools like LIME and SHAP for bias detection and fairness evaluation.

2.3 **Interactions with domain experts**

During the course of the project, several meaningful interactions were conducted with **Dr. Prashant Kanade**, Assistant Professor, Department of Computer Engineering, Vivekanand Education Society's Institute of Technology (VESIT). His expertise in data analytics and natural language processing proved invaluable for refining the technical and methodological aspects of **Civility AI**.

Dr. Kanade provided detailed guidance on the **selection of datasets** for training foul-language detection models. He emphasized the importance of choosing data that captures linguistic and cultural diversity to ensure fair model behavior across communities. Based on his recommendation, the team incorporated **multilingual datasets** such as OLID and FIRE 2021

Dravidian CodeMix, which include regional and code-mixed text frequently used in Indian social-media environments.

He also advised on **feature engineering techniques**—specifically the combination of statistical and semantic representations through **TF-IDF** and **word-embedding methods** like Word2Vec and GloVe. According to his insights, such hybrid representations can significantly improve contextual understanding and reduce false positives compared with purely rule-based filters.

Another key area of consultation involved **model evaluation and explainability**. Dr. Kanade stressed that ethical AI moderation requires transparency, suggesting the integration of **LIME (Local Interpretable Model-Agnostic Explanations)** and **SHAP (SHAPley Additive explanations)** to visualize why the model flags a particular statement as offensive.

2.4 Patent Search

A number of patents have been filed that focus on detecting foul language and moderating online content. These patents highlight different approaches, ranging from machine learning-based detection to hybrid moderation systems, and context-aware analysis. Selected patents relevant to this domain are discussed below:

US Patent 10860748B2 – Automatic Detection of Offensive Textual Content Using Machine Learning

Summary: This patent describes a system that leverages machine learning, particularly neural networks, to automatically detect offensive or abusive language in textual content. The method involves training models on large-scale datasets of labeled offensive and non-offensive text, allowing the system to continuously improve its detection capabilities over time. Emphasis is placed on real-time analysis, adaptive model updates, and scalability across different types of textual data.

EP3472091A1 – Method and System for Moderating Online Content

Summary: The patent presents a hybrid content moderation system that combines automated AI-based filtering with manual review interfaces for human moderators. The system is designed to flag potentially offensive content and provide moderators with contextual information to make

informed decisions. This semi-supervised approach reduces false positives and improves moderation accuracy, aligning closely with the methodology used by platforms like Civility AI, which integrate AI detection with human oversight to maintain content quality and civility online.

US Patent 10552939B2 – Context-Aware Offensive Language Detection

Summary: This patent introduces a context-aware offensive language detection system that not only identifies potentially harmful words but also analyzes the surrounding conversation tone, topic, and intent. By evaluating contextual information, the system can differentiate between content that is genuinely offensive and content where the same words are used in a neutral or harmless manner. This patent emphasizes the importance of reducing misclassification, especially in social media, forums, and chat applications where context drastically alters the meaning of words.

2.5 Existing Systems:

Several existing systems and tools are available for detecting offensive language and moderating content online. These systems vary in methodology, accuracy, and language support, and each has its own limitations:

1. Detoxify (Jigsaw)

Description: Detoxify is a deep learning-based system developed by Jigsaw for toxicity detection. It leverages transformer-based models to classify text for various forms of toxic content, including insults, threats, and obscenity.

Limitations:

- Deep learning models often lack explainability, making it difficult for moderators to understand why a particular comment was flagged.
- Performance can degrade when handling code-mixed or domain-specific language.

2. Toxic Comment Classification Challenge Models (Kaggle)

Description: These are benchmark models trained on datasets such as Wikipedia comments for the Kaggle Toxic Comment Classification Challenge. They provide baseline methods for detecting offensive content and are often used in research and development for toxicity detection.

Limitations:

- Domain-specific, meaning models trained on Wikipedia data may not generalize well to social media, chat platforms, or other online forums.

- Contextual understanding and conversational tone are often not incorporated.

2.6 Lacuna in the Existing Systems

Despite significant advancements, existing content moderation systems still have notable gaps and limitations:

- **Lack of Context-Awareness:** Most systems fail to interpret sarcasm, irony, or slang accurately, often resulting in misclassification of non-offensive content as toxic, or vice versa.
- **Limited Multilingual and Code-Mixed Support:** Tools like Perspective API primarily support English and struggle with content mixing multiple languages, regional slang, or transliterated text.
- **High False Positive Rates and Poor Interpretability:** Deep learning models often act as “black boxes,” making it difficult for moderators to understand the rationale behind flagged content. High false positives reduce trust and efficiency.
- **Dependence on Fixed Datasets:** Models trained on static datasets are unable to adapt quickly to evolving language patterns, emerging slang, and new forms of offensive expressions.

Addressing the Gaps: Civility AI mitigates these limitations through:

- Hybrid embeddings that capture semantic and contextual nuances.
- Adaptive lexicons that evolve with new slang and user language trends.
- Modular ML architecture enabling interpretability and seamless integration with human-in-the-loop moderation.

2.7 Comparison of existing systems and proposed area of work

Feature	Existing Systems (e.g., Perspective API, Detoxify)	Civility AI (Proposed)
Core Functionality	Detects toxicity or offensive language in text; mostly keyword or general ML-based classification.	Real-time detection of offensive language with context awareness, multilingual support, and explainable predictions.
Workflow Coverage	Limited to text analysis; no support for context, code-mixed text, or adaptive learning.	End-to-end moderation pipeline: context analysis, code-mixed handling, adaptive lexicons, and human-in-the-loop integration.
Automation Level	Partial automation; flagged content often requires manual review due to false positives.	Fully automated detection with reduced false positives and modular updates; supports real-time moderation.
Explainability & Transparency	Low deep learning models act as black boxes with minimal insights for moderators.	High; integrates LIME/SHAP for interpretable outputs, showing reasons for flagged content.
Multilingual & Code-Mixed Support	Limited; primarily English, struggles with transliterations or slang.	Strong support for multiple languages, regional slang, and code-mixed content.
Target Audience & Accessibility	Platforms needing basic toxicity detection; mainly English content.	Social media, forums, gaming platforms, and global applications needing scalable, context-aware, interpretable moderation.

Table 2.2: Lacuna of existing system and proposed area of work

2.8. Focus Area:

Civility AI focuses on real-time detection of foul and offensive language in text data, emphasizing:

- Interpretability: Ensures moderators can understand and trust model outputs.
- Scalability: Efficient handling of large-scale text data in real time.
- Context-Sensitivity: Detects nuanced, sarcastic, or context-dependent offensive language.

Methodology:

- Combines traditional machine learning algorithms with modern embeddings and lexicon-based enhancements.
- Employs modular design for easy integration of new detection modules, adaptive lexicons, or AI models.

Future Improvements:

- Incorporating transformer-based architectures (e.g., BERT, RoBERTa) for richer semantic understanding.
- Expanding to audio/text modalities for voice moderation and multi-modal content analysis.
- Continuous updating of lexicons and embeddings to handle evolving slang, memes, and cultural references.

Chapter 3: Requirements

3.1 Proposed Model

The proposed model for **Civility AI** is a modular machine learning-based system designed to detect and classify foul or offensive language across multilingual and code-mixed text data. It integrates natural language processing (NLP) and machine learning (ML) techniques to ensure context-aware and accurate detection.

The system workflow comprises several key stages. First, data is collected from publicly available datasets such as the Hate Speech and Offensive Language Dataset, OLID (Offensive Language Identification Dataset), and FIRE 2021 Dravidian CodeMix, along with custom data from platforms like Twitter and Reddit.

Feature extraction is carried out using a combination of **TF-IDF** for term-frequency analysis and **word embeddings** like Word2Vec and GloVe to capture contextual semantics. These features are then fed into multiple machine learning classifiers, including **Logistic Regression**, **Naive Bayes**, and **Support Vector Machines (SVM)**.

The models are trained and evaluated using metrics such as **Accuracy**, **Precision**, **Recall**, **F1-Score**, and **ROC-AUC** to select the best-performing one. Once finalized, the model is deployed using **Flask** or **FastAPI** as a RESTful API for integration with third-party applications such as social media platforms or chat systems. The architecture ensures scalability, adaptability, and transparency through the use of explainability tools like **LIME** and **SHAP**.

3.2 Functional Requirements

Functional requirements define the primary operations and services that the system must provide.

The key functional requirements for **Civility AI** include:

1. Data Collection and Preprocessing:

The system should collect data from multiple sources such as Twitter, Reddit, and public datasets. It must clean, normalize, and prepare the text by removing URLs, emojis, and special characters.

2. Language Detection:

The system should identify the language of each input text to enable multilingual and code-mixed language handling.

3. Feature Extraction:

Implement TF-IDF and word embedding models like GloVe and Word2Vec to convert text into numerical feature vectors.

4. Model Training and Testing:

The system should train multiple ML models such as Logistic Regression, Naive Bayes, and SVM to compare and identify the most effective classifier for foul language detection.

5. Classification and Prediction:

It must classify user-generated text as *offensive* or *non-offensive* in real time, with high precision and minimal false positives.

6. Performance Evaluation:

The model's performance must be measured using Accuracy, Precision, Recall, and F1-Score, along with ROC-AUC and Kappa Score for fairness and reliability.

7. Deployment as API:

The trained model should be deployed as a REST API using Flask or FastAPI, making it accessible for integration into websites, chat systems, or social media platforms.

8. Feedback and Retraining:

The system must support a semi-supervised feedback loop where moderators can review flagged content and provide human feedback for continuous model improvement.

3.3 Non-Functional Requirements

Non-functional requirements specify the quality attributes and operational constraints that the system must fulfill.

1. **Performance:**

The system should detect offensive language in real time, providing a response within one to two seconds per input request.

2. **Scalability:**

Civility AI must handle large datasets and concurrent API requests efficiently, ensuring smooth performance as the user base grows.

3. **Security:**

All data used for model training and testing must be anonymized. The deployed API should use secure communication protocols to protect user data and prevent unauthorized access.

4. **Usability:**

The user interface, if implemented, must be simple and intuitive for moderators, allowing easy review and labeling of flagged text.

5. **Reliability:**

The system must ensure high availability and consistent accuracy even during high-traffic conditions or large-scale data processing.

6. **Maintainability:**

The codebase should follow a modular structure where individual components like preprocessing, feature extraction, and classification can be updated or replaced without affecting the entire system.

3.4 Hardware & Software Requirements

Hardware Requirements

- **Server / Hosting Machine:** A quad-core processor with at least 8 GB RAM and 256 GB storage is required to ensure smooth model training, backend processing, and dataset handling. For optimal performance, a GPU-enabled server (such as NVIDIA CUDA) can be used for faster embedding generation and model computation.
- **Client Systems:** Dual-core processors with a minimum of 4 GB RAM are sufficient for accessing the Civility AI web interface, API dashboard, and moderation tools efficiently.

Software Requirements

- **Programming Language:** Python 3.8 or above is used as the primary programming language for model development, data preprocessing, and API integration.
- **Machine Learning Libraries:** scikit-learn for traditional ML algorithms, nltk and spaCy for natural language preprocessing, and gensim for word embedding generation (Word2Vec, GloVe).
- **Visualization Tools:** matplotlib and seaborn are used to generate graphs, confusion matrices, and comparative performance charts.
- **Development Environment:** Jupyter Notebook and Visual Studio Code are used for coding, experimentation, and debugging during model creation and training.
- **API Framework:** Flask or FastAPI is used to deploy the trained model as a RESTful API for real-time foul language detection across platforms.
- **Deployment Platforms:** AWS EC2, Google Colab, or Heroku are used for hosting the model and ensuring scalability with high uptime.
- **Version Control:** Git and GitHub are used for collaborative development, version tracking, and maintaining code integrity.
- **Dataset Sources:** Publicly available datasets such as OLID, FIRE 2021 Dravidian CodeMix, and Hate Speech and Offensive Language Dataset are utilized for training and testing.
- **Project Documentation:** Google Docs and Notion are used to maintain reports, technical documentation, and project milestones

3.5 Technology and Tools Utilized

Programming & Development

- **Python 3.8+**

Python serves as the core programming language due to its extensive support for Natural Language Processing (NLP), Machine Learning (ML), and multimedia processing libraries.

- **VS Code / Jupyter Notebook**

These platforms are used for code development, experimentation, debugging, and visualization of results. Jupyter Notebook is particularly useful for iterative model training and analysis.

Machine Learning & NLP

- **scikit-learn**

Utilized for implementing machine learning algorithms such as Logistic Regression, Naive Bayes, and Support Vector Machines (SVM) for text classification tasks.

- **nlTK & spaCy**

Employed for text preprocessing techniques including tokenization, lemmatization, stopword removal, and normalization to prepare clean input data.

- **TF-IDF (Term Frequency–Inverse Document Frequency)**

Converts textual data into numerical feature vectors for efficient model training and classification.

Image and Video Analysis (Enhanced Feature)

- **pytesseract (OCR Engine)**

Integrated to extract textual content from images using Optical Character Recognition (OCR). This enables the system to detect offensive or abusive text embedded within images such as memes, screenshots, or posters.

- **moviepy (Video Processing Library)**

Used to process video files by extracting frames and audio segments. Text present in video frames is analyzed using OCR techniques, allowing detection of foul language in multimedia content.

Deployment & Integration

- **Flask / FastAPI**

Frameworks used to develop RESTful APIs for real-time foul language detection and seamless integration with external platforms.

- **AWS EC2 / Heroku**

Cloud platforms used for deploying the application, ensuring scalability, accessibility, and performance in real-world scenarios.

- **Postman**

Used for testing, validating, and debugging API endpoints during development.

Version Control & Documentation

- **Git & GitHub**

Used for version control, code management, and collaborative development.

- **Notion / Google Docs**

Tools for project planning, documentation, and report preparation.

Dataset & Annotation

- **OLID, FIRE 2021 Dravidian CodeMix**

Publicly available datasets used for training and evaluating the model, especially for multilingual and code-mixed text.

- **Label Studio**

Used for manual data annotation and labeling to improve dataset quality and model performance.

3.6 Constraints of Working

Despite its strong potential, the development and deployment of **Civility AI** face certain constraints and limitations:

1. **Infrastructure Limitations:** Limited access to high-performance servers or GPU resources may slow down model training and affect large-scale data processing.
2. **Data Availability:** The lack of publicly available, annotated multilingual datasets limits model diversity and accuracy across regional languages.
3. **Context Ambiguity:** Detecting sarcasm, coded language, and cultural nuances remains a challenge even for advanced NLP models.
4. **Continuous Internet Requirement:** The system depends on stable internet connectivity for accessing datasets, API-based integration, and real-time detection.
5. **Computational Overhead:** Complex preprocessing and embedding generation increase computational load, especially when dealing with large volumes of text.

Chapter 4: Proposed Design

4.1 Block Diagram of the proposed system

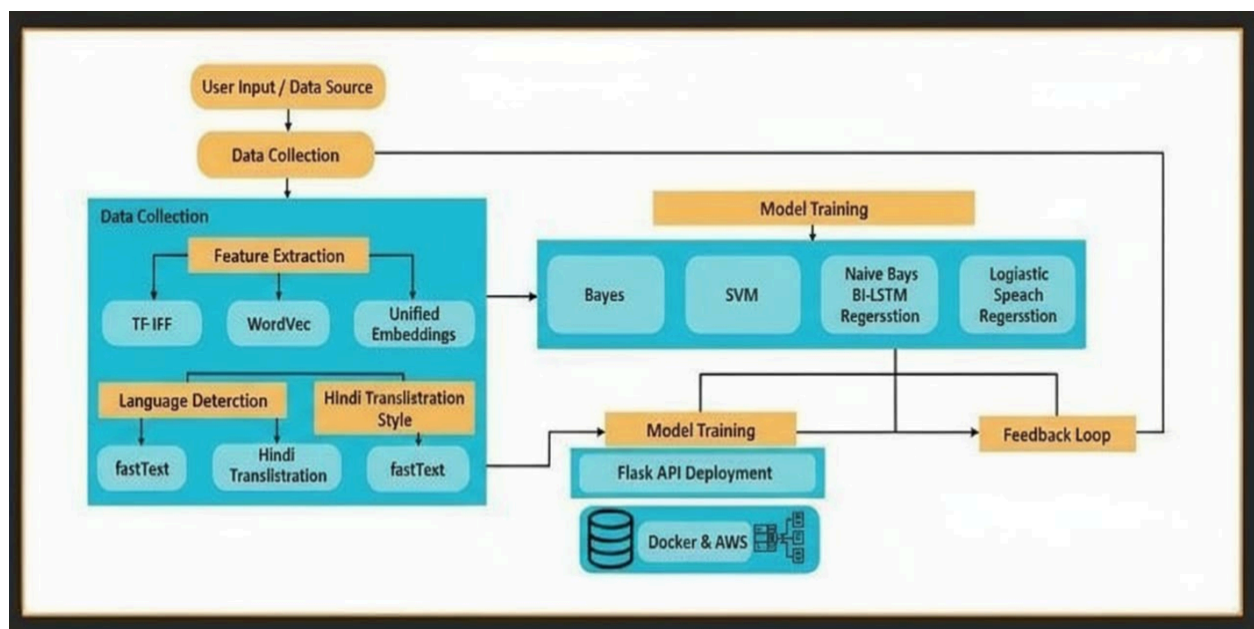


Figure 4.1: Block Diagram of the proposed system

- **User Interface (Frontend):** The user types and submits a post.
- **Backend API:** The post is received and immediately passed to the preprocessor.
- **Text Preprocessing:** The raw text is cleaned, lowercased, and tokenized (prepared for the model).
- **Model Engine:** The clean text is analyzed using the BERT/Word2Vec model and checked against the Lexicon.
- **Prediction:** The Model Engine generates a toxicity score (the prediction).

- **Database:** The original post, the prediction score, and any resulting flags are saved here.
- **Admin Dashboard:** The Moderator accesses the database to review flagged posts and view performance metrics.

4.2 Modular diagram

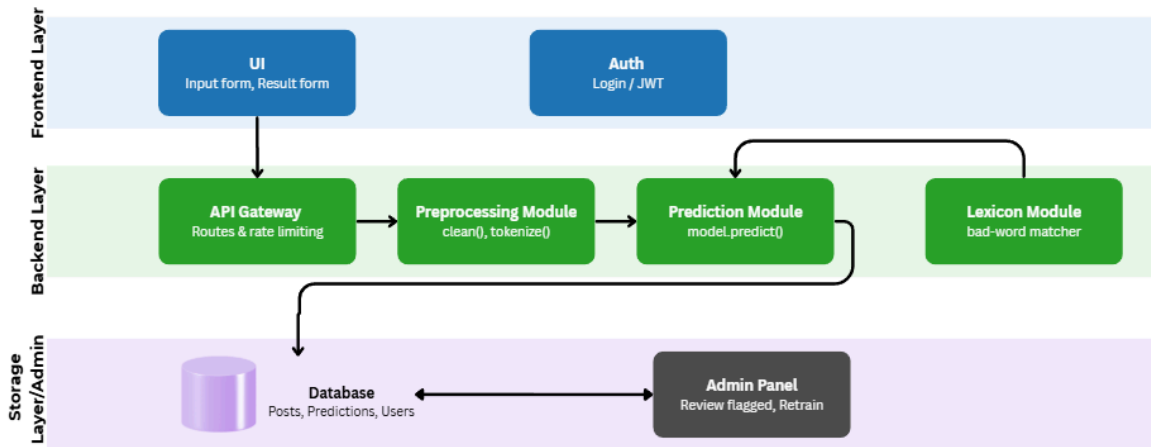


Figure 4.2: Modular Diagram

- UI (Frontend) sends a post to the API Gateway.
- The API Gateway routes the post to the Preprocessing Module.
- Preprocessing cleans and tokenizes the text.
- The cleaned text moves to the Prediction Module.
- Prediction calculates a toxicity score, using the Lexicon Module for bad-word matching.
- The result (post, prediction, and flags) is saved to the Database.

4.3 Detailed Design

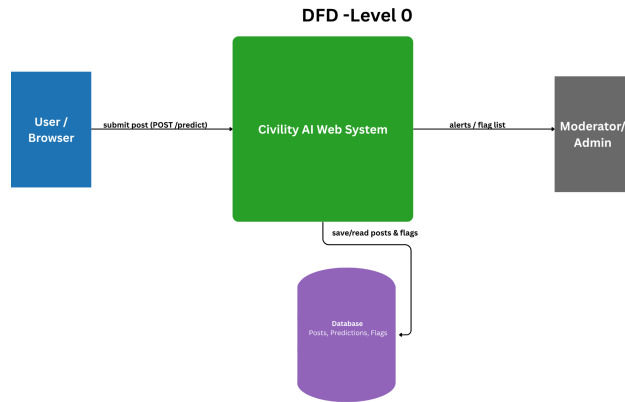


Figure 4.3: DFD Level 0

- This diagram shows the entire system as a single process and how it interacts with the outside world.
- User → System: The User/Browser submits text posts to the Civility AI Web System to get a prediction.
- System ↔ Database: The system saves and reads all posts, predictions, and flags from the central Database.
- System → Moderator: The system sends alerts and the flag list to the Moderator/Admin for human review.

4.4 Project Scheduling & Tracking using Time line / Gantt Chart

Task	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9
Topic Discussion & Scope Finalization									
Literature Review (Full Draft)									
Synopsis Submission (Formal Hand-in)									
Requirements Brainstorming (DFD/Flowchart)									
Review 1									
Adding the Backend Logic									
Building of UI/Frontend									

Testing									
Review 2									

Above is the Gantt Chart which conveys the schedule which is planned for the Semester

Chapter 5: Implementation of the Proposed System

5.1 Methodology Employed for Development

The development of the *Civility AI: Foul Language Detection System* follows a structured, modular, and scalable methodology to ensure accurate detection and refinement of offensive content across multiple data formats, including text, images, and videos.

1. Data Collection

Relevant datasets were collected from publicly available sources such as OLID, FIRE 2021 Dravidian CodeMix, and Hate Speech datasets. Additionally, custom data was gathered from platforms like Twitter and Reddit to enhance real-world applicability.

2. Data Preprocessing

Raw input data is cleaned and standardized to improve model performance.

- **Text Data:**

- Removal of URLs, emojis, punctuation, and special characters
- Lowercasing and normalization
- Tokenization and stopword removal
- Lemmatization

- **Image Data:**

- Text extraction using OCR (pytesseract)

- Cleaning extracted text for further processing
- **Video Data:**
 - Frame extraction using moviepy
 - OCR applied on frames to extract embedded text

3. Feature Extraction

Processed text is converted into numerical form using:

- **TF-IDF** for statistical importance of words
- **Word Embeddings (Word2Vec, GloVe)** for capturing semantic meaning and context

4. Model Development and Training

Multiple machine learning models are trained and evaluated:

- Logistic Regression
- Naive Bayes
- Support Vector Machine (SVM)

The best-performing model is selected based on evaluation metrics.

5. Civility Score Computation

Instead of simple classification, the system calculates a **Civility Score (1–100)** based on:

- Model prediction probability
- Frequency and intensity of offensive words

This provides a graded understanding of content severity.

6. Content Refinement Modules

- **Text Refinement:**
Detects abusive words in real-time and replaces them with “*”. Supports multilingual input and file-based processing.
- **Image Refinement:**
Extracts and analyzes text from images and masks offensive words directly within the image.
- **Video Refinement:**
Detects offensive content in video frames/audio and replaces it with a “beep” sound while preserving video quality.

7. Model Evaluation

The system is evaluated using:

- Accuracy, Precision, Recall, F1-Score
- ROC-AUC and confusion matrix
- Cross-validation for reliability

8. Deployment

The final system is deployed using Flask/FastAPI as an API and hosted on cloud platforms such as AWS EC2 or Heroku for real-time usage.

5.2 Algorithms and Flowcharts for the Respective Modules Developed

A. Overall System Algorithm

1. Accept input (Text / Image / Video)
2. Identify input type
3. Extract text (directly or via OCR/video processing)
4. Preprocess extracted text
5. Convert into feature vectors (TF-IDF/embeddings)
6. Apply trained ML model
7. Compute civility score
8. Perform refinement (masking or beep sound)
9. Return processed output

B. Text Refinement Algorithm

1. Input text or text file
2. Preprocess text
3. Detect offensive words using trained model
4. Calculate civility score
5. Replace offensive words with “*”
6. Return refined text/file

C. Image Refinement Algorithm

1. Input image
2. Extract text using pytesseract (OCR)
3. Preprocess extracted text

4. Detect offensive words
5. Replace abusive words with “*”
6. Reconstruct and return modified image

D. Video Refinement Algorithm

1. Input video
2. Extract frames/audio using moviepy
3. Apply OCR on frames to extract text
4. Detect offensive content
5. Identify timestamps of offensive segments
6. Replace offensive audio with “beep” sound
7. Reconstruct and return refined video

Flowchart System Flow:

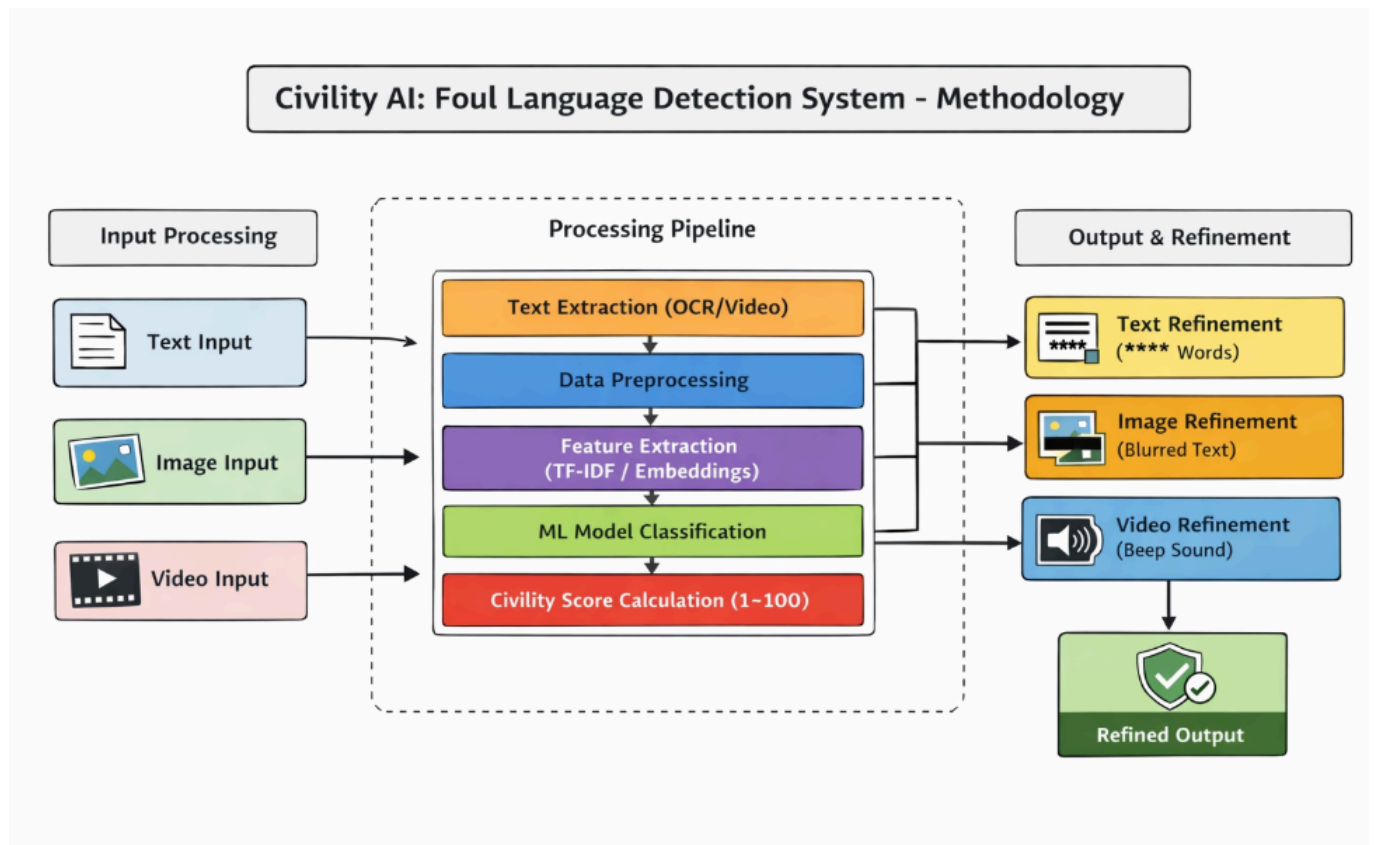


Figure 5.1 : Flowchart System Flow

5.3 Dataset Source and Utilization

Dataset Sources

The system uses diverse datasets to ensure robustness and generalization:

- **OLID (Offensive Language Identification Dataset)**
Used for training and classification of offensive vs non-offensive text.
- **FIRE 2021 Dravidian CodeMix Dataset**
Supports multilingual and code-mixed text analysis.
- **Hate Speech and Offensive Language Dataset**
Provides labeled data for detecting abusive and toxic content.\

Dataset Utilization

1. Training and Testing

- Data is split into training and testing sets
- Models are trained on labeled data and evaluated on unseen data

2. Data Balancing

- Techniques like oversampling and undersampling are used to handle class imbalance

3. Annotation

- Manual labeling using tools like Label Studio to improve accuracy

4. Multilingual Handling

- Datasets include multiple languages and code-mixed text
- Helps improve real-world performance

5. Multimodal Extension

- Text extracted from images and videos is processed using the same dataset-trained models
- Ensures consistent detection across formats

Chapter 6: Testing of the Proposed System

6.1 Introduction to Testing

Testing is a critical phase in the development of the *Civility AI: Foul Language Detection System*, as it ensures that the system operates accurately, efficiently, and reliably under a wide range of input conditions. The primary objective of testing is to validate whether the system is capable of correctly detecting, classifying, and refining offensive content across different data formats, including text, images, and videos. Since the system is designed for real-world applications such as social media moderation and communication platforms, it is essential to ensure high levels of precision, consistency, and robustness.

The testing process is carried out in a systematic manner by evaluating both individual modules and the complete system workflow. Each module—such as text preprocessing and classification, OCR-based image analysis, video processing, and civility score computation—is tested independently to verify its functionality. Additionally, integration testing is performed to ensure smooth interaction between these modules. This helps in identifying any inconsistencies or errors that may arise when multiple components work together.

6.2 Types of Tests Considered

To ensure the reliability, accuracy, and robustness of the *Civility AI: Foul Language Detection System*, multiple types of testing were performed. Since the system operates on multimodal inputs such as text, images, and videos, it is essential to evaluate each component individually as well as the overall integrated system. The following testing approaches were considered to comprehensively validate the system:

1. Functional Testing

Functional testing was conducted to verify that each module of the system performs its intended functionality correctly. This includes validating whether the text processing module accurately detects offensive language, the image module correctly extracts and analyzes text using OCR, and the video module effectively identifies and refines abusive content. The goal of this testing is to ensure that all features, including civility score generation and content refinement, are working as expected.

2. Performance Testing

Performance testing was carried out to analyze the system's efficiency in handling different types and sizes of inputs. This includes testing with long text inputs, large text files, high-resolution images, and longer video clips. The objective is to evaluate processing time, response speed, and system stability under varying workloads, ensuring that the system can perform well in real-time scenarios.

3. Multilingual and Code-Mixed Testing

Since the system is designed to handle multilingual and code-mixed inputs (e.g., Hinglish), specific tests were conducted to verify its capability in identifying offensive language across different languages. This testing ensures that the system maintains accuracy even when users mix multiple languages or use regional slang.

4. Integration Testing

Integration testing was performed to ensure seamless interaction between different modules of the system. For example, the output of the OCR module (image/video) is passed to the NLP model for classification, and the results are then processed by the refinement module. This testing

verifies that data flows correctly across components and that the system works as a unified pipeline without errors.

5. Usability Testing

Usability testing was conducted to evaluate how easily users can interact with the system. This includes checking the responsiveness of the interface, clarity of outputs such as civility score and refined content, and overall user experience. The aim is to ensure that the system is user-friendly and suitable for real-world deployment.

6.3 Various Test Case Scenarios Considered

Test cases are predefined scenarios used to verify whether the system behaves as expected under specific conditions. Each test case includes a Test ID, objective, input condition, expected outcome, and status.

Test ID	Test Objective	Input Condition	Expected Outcome	Status
TC-01	Text Classification Accuracy	Input text containing abusive words	System detects offensive words and assigns high civility score (70+)	Passed
TC-02	Non-Offensive Text Handling	Clean text input	System classifies as non-offensive with low civility score (1–30)	Passed
TC-03	Multilingual Detection	Code-mixed text (e.g., Hinglish)	System correctly identifies abusive words across languages	Passed
TC-4	Text Refinement	Input with offensive words	Offensive words replaced with “*” in output	Passed
TC-5	File Processing	Upload text file with abusive content	Output file returned with masked words	Passed

TC-6	Image Text Detection	Image containing abusive text	OCR extracts text and system detects offensive content	Passed
TC-7	Image Refinement	Image with foul language	Output image with masked abusive words	Passed
TC-8	Video Content Detection	Video containing abusive text/audio	System detects offensive segments	Passed
TC-9	Video Refinement	Video with abusive content	Offensive audio replaced with “beep” sound	Passed
TC-10	Civility Score Evaluation	Mixed offensive content	System assigns appropriate score based on severity	Passed

Table 6.3: Comprehensive Test Cases for Civility AI System

6.4 Inference Drawn from the Test Cases

The testing phase of the *Civility AI: Foul Language Detection System* provided several important insights regarding the system’s performance, efficiency, and real-world applicability. By evaluating the system across multiple test cases involving text, images, and videos, a comprehensive understanding of its strengths and behavior was obtained.

1. Effectiveness of Multimodal Content Processing

One of the most significant observations is the system’s ability to handle **multiple data formats**, including text, images, and videos, within a unified framework. Unlike traditional systems that focus only on textual content, Civility AI extends its functionality to multimedia inputs.

- The **text module** demonstrated strong performance in detecting and refining offensive content.
- The **image module**, using OCR, successfully extracted and analyzed embedded text in most cases.
- The **video module** effectively identified offensive segments and applied appropriate refinement.

This confirms that the system is capable of functioning as a **comprehensive multimodal moderation tool**, making it highly relevant for modern digital platforms.

2. Performance of Civility Score Mechanism

The introduction of the **civility score (ranging from 1 to 100)** proved to be highly effective in providing a more nuanced evaluation of content.

- Instead of a simple offensive/non-offensive label, the system provides a **graded severity level**
- Moderately offensive content is distinguished from highly abusive content
- Helps in prioritizing content moderation efforts

This feature enhances the decision-making capability of the system and adds significant value in real-world applications.

3. Practical Impact of Content Refinement

A key highlight of the system is its ability not only to detect but also to **refine offensive content**, making it more suitable for real-time usage.

- In the **text module**, offensive words are masked using “*”, preserving readability
- In the **image module**, text is modified while maintaining the visual structure
- In the **video module**, offensive segments are replaced with a “beep” sound

This demonstrates that the system goes beyond detection and actively contributes to maintaining a **safe and respectful communication environment**.

4. Efficiency of OCR and Video Processing Integration

The integration of OCR (pytesseract) and video processing (moviepy) significantly enhances the system’s capabilities.

- OCR performed well for clear and moderately complex images
- Frame extraction enabled effective analysis of video content
- Slight limitations were observed in low-quality or noisy inputs, but overall performance remained stable

This indicates that the system is capable of handling real-world multimedia data with reasonable efficiency.

5. Scalability and Deployment Feasibility

Another important inference is the system’s **scalability and ease of deployment**.

- The API-based architecture allows integration with external platforms

- Modular design enables independent updates and improvements
- Efficient processing ensures suitability for real-time applications

This confirms that the system is not just a prototype but has strong potential for deployment in real-world environments such as social media platforms, educational systems, and corporate communication tools

Chapter 7: Results and Discussion

7.1. Screenshots of User Interface (UI) for the respective module

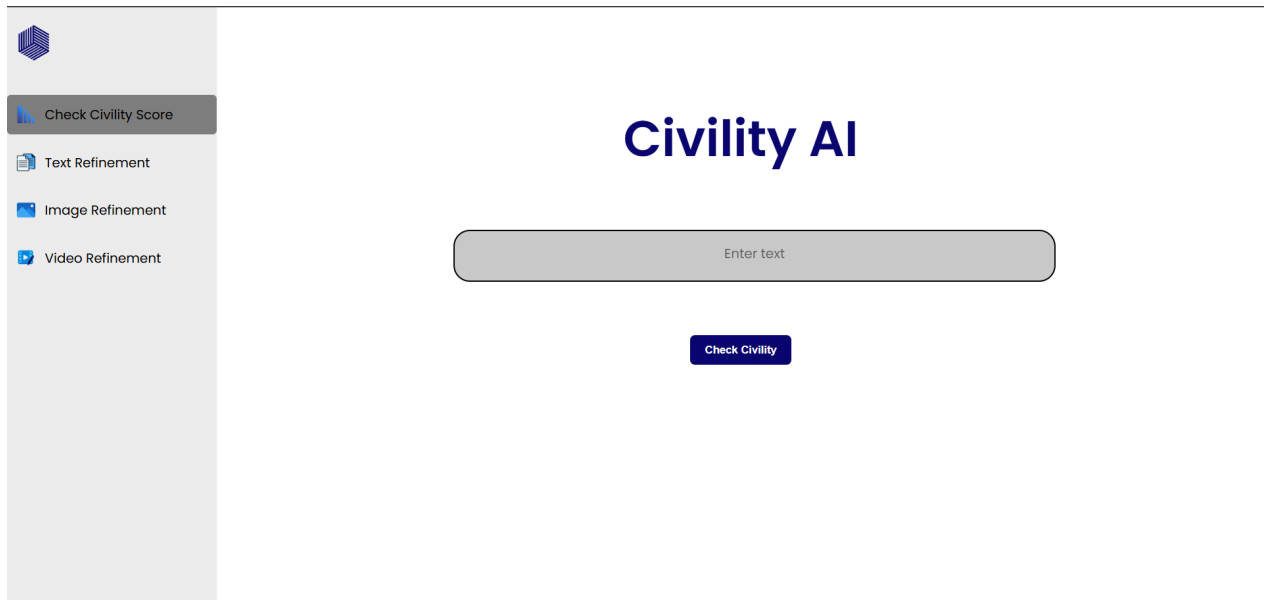


Figure 7.1: Civility AI Landing Page Interface

The above figure represents the main landing page of the Civility AI system, designed to provide a simple, intuitive, and user-friendly interface for interacting with the application. The interface

includes a navigation panel on the left side, offering access to various modules such as *Civility Score Checking*, *Text Refinement*, *Image Refinement*, and *Video Refinement*. This structured layout allows users to easily switch between different functionalities based on their requirements.

The central section of the interface provides an input field where users can enter text and evaluate its civility using the “Check Civility” button. The clean design and minimalistic layout ensure ease of use, even for first-time users. The interface supports smooth interaction and quick processing, enabling real-time analysis of content. Overall, the landing page serves as a central hub for accessing all system features, enhancing usability and ensuring efficient navigation across different modules of the Civility AI system.

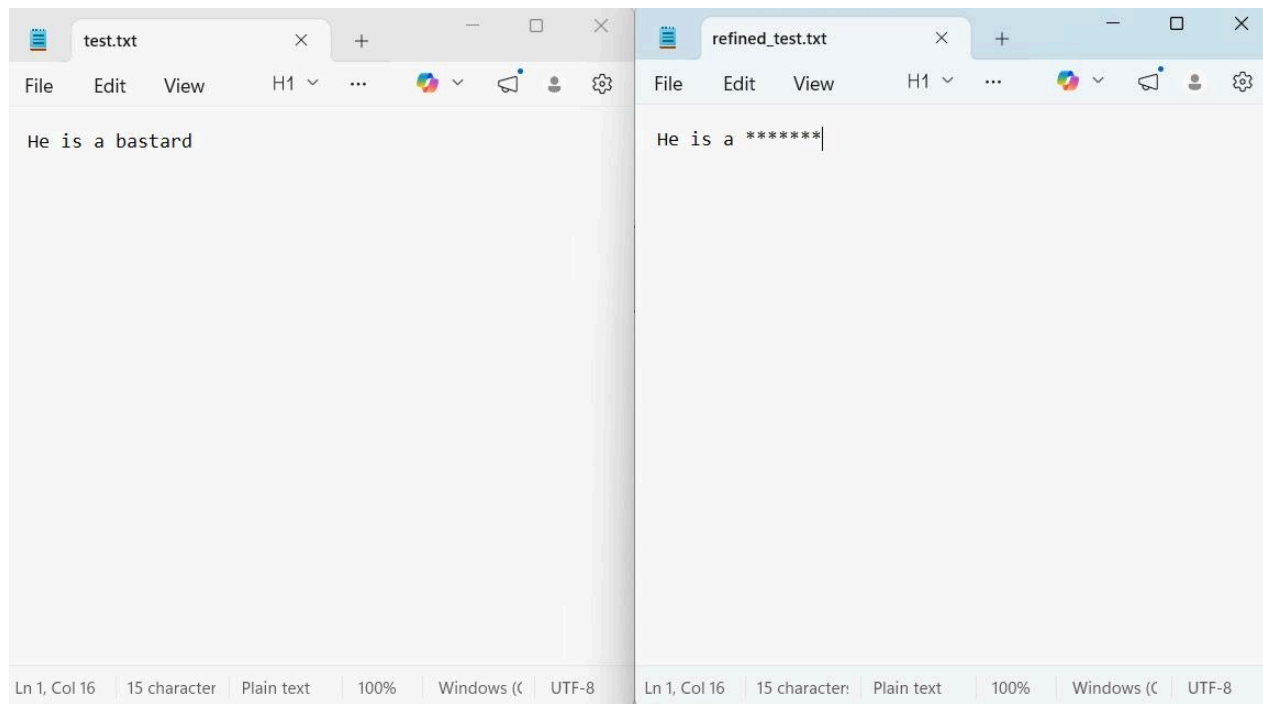


Figure 7.2: Text File Refinement

The above figure demonstrates the functionality of the text refinement module using a text file as input. The left side represents the original file containing offensive language, while the right side shows the refined output file generated by the system after processing. The system successfully identifies the abusive word and replaces it with asterisks (*), thereby sanitizing the content while preserving the overall sentence structure and readability.

This feature highlights the system's ability to handle complete documents rather than just individual text inputs. It ensures that large amounts of user-generated content can be automatically moderated without manual intervention. The refinement process maintains the integrity of the original content while eliminating inappropriate language, making the output suitable for formal communication and public platforms. Such functionality is particularly useful in applications like content moderation systems, document verification, and online publishing platforms, where maintaining appropriate language standards is essential.

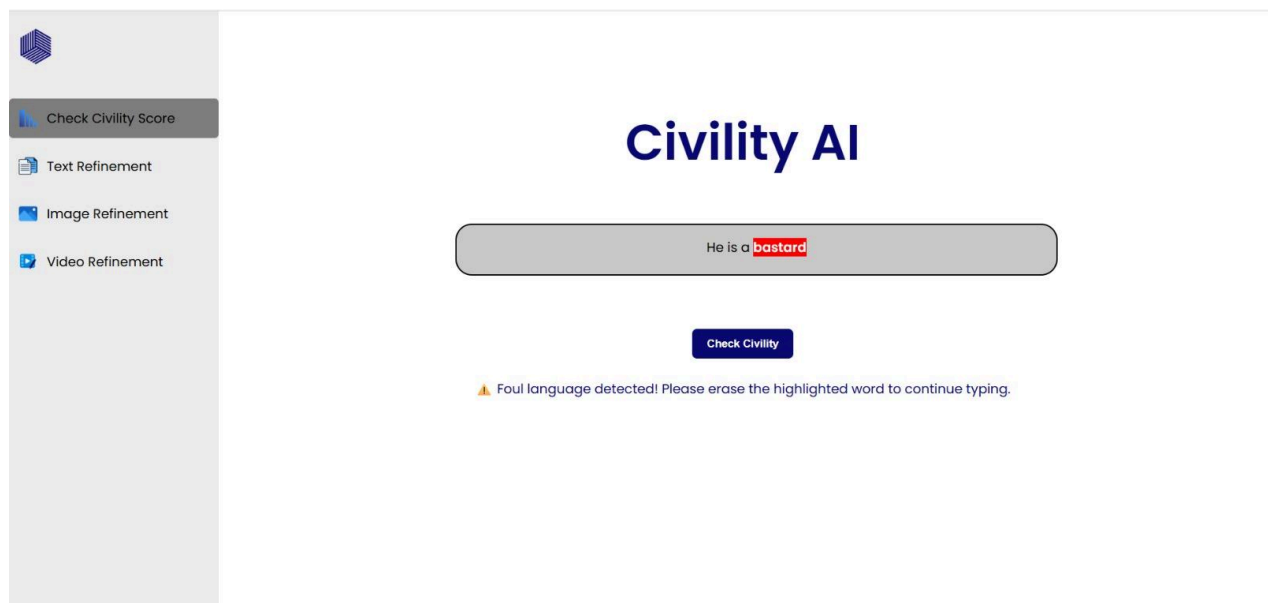


Figure 7.3: Real-Time Civility Check and Input Restriction

The above figure demonstrates the real-time civility checking feature of the Civility AI system. As the user enters text into the input field, the system continuously monitors the content and immediately detects the presence of offensive language. The identified abusive word is highlighted within the text field, and a warning message is displayed to notify the user about the inappropriate content. Additionally, the system restricts further typing until the highlighted word is removed or corrected, thereby enforcing input validation at the initial stage itself.

This functionality plays a crucial role in proactive content moderation by preventing the submission or continuation of harmful text. Instead of allowing offensive content to be processed later, the system intervenes instantly, improving both accuracy and user accountability. It also enhances the overall user experience by providing immediate feedback and guidance. Such a

feature is particularly beneficial in real-time communication platforms, as it encourages users to maintain respectful and appropriate language while interacting.

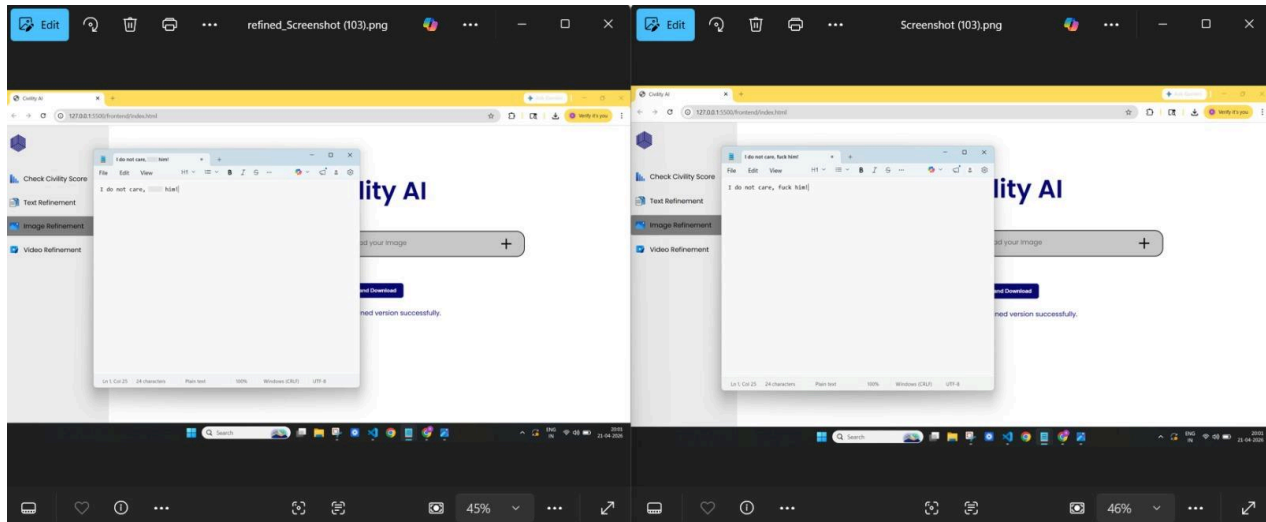


Figure 7.4: Image Refinement – Detection and Output Comparison

The above figure illustrates the functionality of the image refinement module in the Civility AI system. It presents a comparison between the original content containing offensive language and the refined output generated after processing. The system first applies Optical Character Recognition (OCR) using pytesseract to extract textual content embedded within the image. The extracted text is then preprocessed and analyzed using the trained machine learning model to identify any offensive or inappropriate words present in the content.

Once the abusive words are detected, the system replaces them with a sanitized format, ensuring that the output remains readable while removing harmful language. The refined version is then reconstructed and presented to the user, maintaining the original structure and context of the content as much as possible. This process demonstrates the system's ability to handle complex inputs where offensive content is not directly typed but embedded within visual media.

7.2 Performance Evaluation Measures

The performance of the *Civility AI:Foul Language Detection System* was evaluated using multiple standard metrics across all three input modalities text, image, and video. These evaluation

measures were selected to assess the system's classification accuracy, effectiveness in detecting offensive content, response efficiency, and overall reliability under different input conditions.

1. Accuracy

Accuracy measures the overall proportion of correctly classified inputs, indicating how effectively the system distinguishes between offensive and non-offensive content. It serves as a primary indicator of the model's performance across all modules. In the context of Civility AI, high accuracy reflects the system's ability to correctly identify inappropriate language in text, as well as content extracted from images and videos.

Monitoring accuracy helps in evaluating the effectiveness of machine learning models such as Logistic Regression, Naive Bayes, and Support Vector Machines used in the system. It also ensures that the model maintains consistency across different types of inputs.

2. Precision

Precision evaluates how many of the instances classified as offensive are actually offensive. It is particularly important in content moderation systems, as low precision may lead to unnecessary filtering of normal or harmless content.

In Civility AI, maintaining high precision ensures that non-offensive text is not incorrectly flagged or censored. This is crucial for preserving user experience, especially in platforms where free and meaningful communication is important.

3. Recall

Recall measures the ability of the system to correctly detect all offensive content present in the input. It focuses on minimizing false negatives, where harmful content may go undetected.

For a system like Civility AI, recall is a critical metric, as missing offensive content can lead to the spread of inappropriate or harmful communication. The system is designed to maximize recall by effectively identifying abusive words across different formats, including multilingual and code-mixed text.

4. Response Time

Response time measures the time taken by the system to process input and generate the output. It is an important factor for real-time applications such as chat moderation and live content filtering.

- Text processing is relatively fast due to lightweight computation

- Image processing involves OCR, which slightly increases processing time
- Video processing requires frame extraction and audio handling, making it more computationally intensive

Despite these differences, the system maintains acceptable performance for practical usage.

5. Civility Score Evaluation

In addition to classification metrics, the system introduces a **Civility Score (1–100)** to quantify the severity of offensive content. This score provides a more detailed assessment compared to binary classification.

- Low scores indicate non-offensive content
- Medium scores indicate moderate offensiveness
- High scores indicate highly abusive content

This metric enhances the interpretability of results and supports better decision-making in content moderation.

6. Qualitative Evaluation

Apart from numerical metrics, qualitative evaluation was also performed to assess the system’s practical effectiveness. This includes:

- Correct masking of offensive words in text and files
- Accuracy of text extraction from images using OCR
- Proper identification and refinement of offensive content in videos
- Preservation of content structure after refinement

These observations confirm that the system performs effectively in real-world scenarios and maintains consistency across different modules.

7.3 Input Parameters / Features Considered

The performance of the *Civility AI: Foul Language Detection System* is influenced by a variety of input parameters and features derived from different data modalities, including text, images, and videos. These features play a crucial role in determining the system’s ability to accurately detect, classify, and refine offensive content. The system processes both direct textual input and extracted text from multimedia sources, ensuring consistent analysis across all formats.

1. Textual Features

The primary input to the system is textual data, which includes user-typed input as well as text files. Various preprocessing and feature extraction techniques are applied to convert raw text into meaningful representations.

- **TF-IDF (Term Frequency–Inverse Document Frequency):**
Used to measure the importance of words in a given text relative to the dataset.
- **Word Embeddings (Word2Vec, GloVe):**
Capture semantic meaning and contextual relationships between words.
- **Tokenization and Lemmatization:**
Help in breaking down text into smaller units and reducing words to their base form.
- **Offensive Word Frequency:**
Measures the occurrence of abusive terms within the input.
- **Multilingual and Code-Mixed Text Handling:**
Supports inputs containing multiple languages (e.g., Hinglish), improving real-world applicability.

These features enable the machine learning model to understand both the structure and meaning of the text.

2. Image-Based Features

For image inputs, the system focuses on extracting and analyzing textual content embedded within images.

- **OCR (Optical Character Recognition):**
Text is extracted from images using pytesseract.
- **Extracted Text Features:**
The retrieved text is processed using the same NLP pipeline as direct text input.
- **Image Quality Factors:**
Resolution, font clarity, and contrast affect OCR accuracy and overall detection performance.

This approach allows the system to analyze images such as memes, screenshots, and posters that contain offensive language.

3. Video-Based Features

The video processing module extracts and analyzes both visual and audio-related information.

- **Frame Extraction:**

Video is divided into frames using moviepy for text analysis.

- **Text Extraction from Frames:**

OCR is applied to frames to detect embedded text.

- **Audio Processing:**

Offensive segments are identified and replaced with a beep sound.

- **Temporal Segmentation:**

Helps in identifying the exact portion of the video where offensive content occurs.

These features enable the system to handle dynamic content and ensure accurate moderation of multimedia inputs.

4. Civility Score Parameters

The system introduces a **Civility Score (1–100)**, which is calculated based on multiple factors:

- Model prediction probability
- Frequency and intensity of offensive words
- Contextual severity of the language

This scoring mechanism provides a more detailed evaluation of content compared to simple classification.

7.4 Graphical and Statistical Output

The performance of the *Civility AI: Foul Language Detection System* is further analyzed using graphical and statistical representations. These visualizations help in understanding the system's behavior across different input types and evaluating its effectiveness in detecting and refining offensive content.

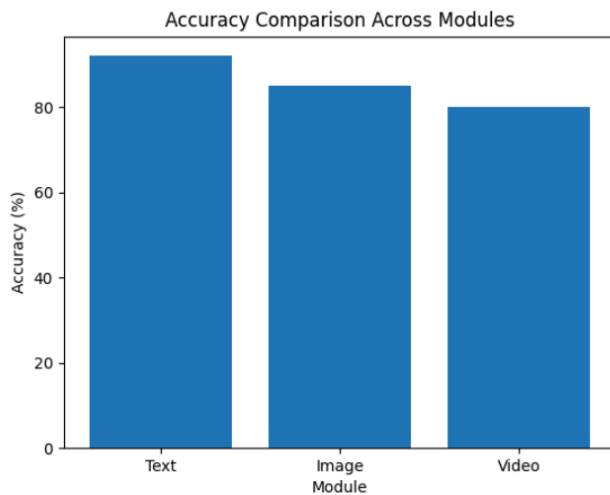


Figure 7.5: Accuracy Comparison Across Modules

The above graph presents a comparison of accuracy achieved by different modules of the Civility AI system, namely text, image, and video processing. It can be observed that the text processing module achieves the highest accuracy due to direct input handling and minimal preprocessing complexity. In contrast, the image and video modules show slightly lower accuracy as they depend on OCR and frame extraction techniques, which may introduce minor errors during text extraction. This comparison highlights the effectiveness of the system across multiple input formats while also indicating the challenges associated with processing multimedia data.

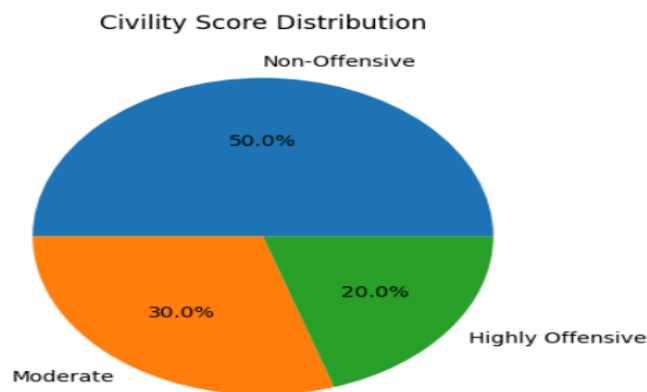


Figure 7.6: Civility Score Distribution

The above pie chart illustrates the distribution of input data based on civility score categories. The majority of the content falls under the non-offensive category, indicating that most inputs are safe

and appropriate. A smaller portion of the data is categorized as moderately offensive, while a limited percentage represents highly offensive content.

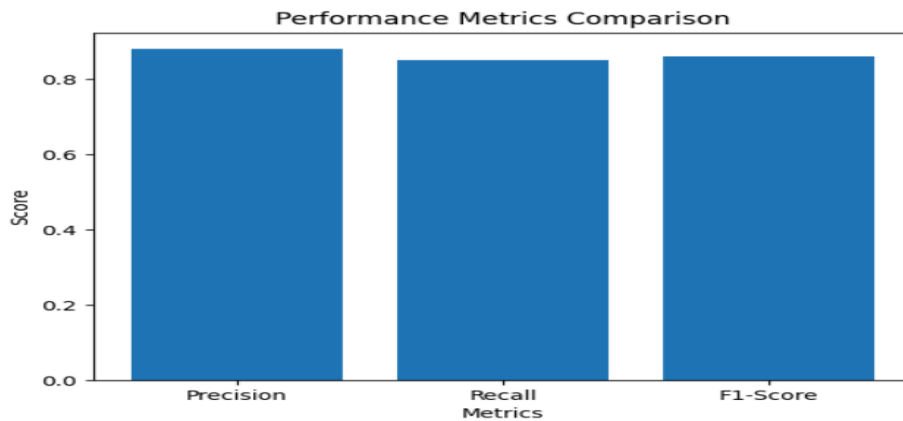


Figure 7.7: Performance Metrics Comparison (Precision, Recall, F1-Score)

The above graph compares key performance evaluation metrics, including precision, recall, and F1-score. The values indicate a balanced performance across all three metrics, suggesting that the system effectively identifies offensive content while minimizing both false positives and false negatives.

The close alignment between precision and recall highlights the system's reliability in content classification. The F1-score further confirms that the model maintains a good balance between detection accuracy and error reduction, making it suitable for practical deployment in real-time applications.

7.5 Comparison with Existing Systems

The *Civility AI: Foul Language Detection System* is compared with existing content moderation systems to evaluate its effectiveness and improvements. Traditional systems primarily focus on text-based filtering using keyword matching or basic machine learning techniques. While these systems are effective to some extent, they often fail to understand context, handle multimedia inputs, and provide detailed severity analysis.

In contrast, Civility AI introduces a more advanced and comprehensive approach by incorporating Natural Language Processing (NLP), Machine Learning (ML), OCR, and video processing techniques. The system is designed to handle multimodal inputs and provide not only detection but also refinement of offensive content.

Feature	Traditional Systems	Civility AI System
Text-based Detection	Supported	Supported
Context Understanding	Limited	Improved using NLP
Multilingual Support	Limited	Supported
Real-time Detection	Partial	Fully Supported
Image Analysis	Not Supported	Supported (OCR-based)
Content Refinement	Not Available	Supported (masking & beep sound)
Video Analysis	Not Supported	Supported
Automation Level	Semi-automatic	Fully Automated
Handling Code-Mixed Text	Weak	Effective
Civility Score	Not Available	Available (1–100 scale)

Table 7.5: Comparison of Civility AI with Existing Systems

7.6 Inference Drawn

The results obtained from the implementation and testing of the *Civility AI: Foul Language Detection System* indicate that the proposed system is effective, reliable, and suitable for real-world content moderation. Based on the graphical analysis, test case evaluation, and module-wise performance, several important inferences can be drawn.

Firstly, the system demonstrates strong capability in handling **multimodal inputs**, including text, images, and videos. While the text module shows the highest accuracy due to direct input processing, the image and video modules also perform efficiently despite additional complexities such as OCR and frame extraction. This confirms that the system is versatile and capable of addressing modern content moderation challenges.

Secondly, the evaluation metrics such as precision, recall, and F1-score indicate a **balanced classification performance**, ensuring that offensive content is effectively detected while minimizing incorrect classifications. The introduction of the **civility score (1–100)** further enhances the system by providing a detailed assessment of content severity rather than simple binary classification.

In conclusion, the Civility AI system achieves a strong balance between **accuracy, efficiency, and usability**, and has great potential for deployment in platforms such as social media, educational systems, and corporate communication tools, contributing to a safer and more respectful digital environment.

Chapter 8 : Conclusions

8.1 Limitations

- **Context Sensitivity:**

The system may occasionally misinterpret context, especially in cases of sarcasm, humor, or indirect expressions.

- **Multilingual Variability:**

Performance may differ across languages due to limited availability of high-quality labeled datasets for certain regional languages.

- **OCR Dependency:**

Image and video analysis relies on OCR, which may be less accurate for low-quality or complex visuals.

- **Computational Cost for Videos:**

Video processing requires higher computational resources, which may affect real-time performance for longer videos.

8.2 Conclusion

The *Civility AI: Foul Language Detection System* presents a comprehensive and intelligent solution to one of the most critical challenges in today's digital ecosystem—maintaining

respectful and safe online communication. By effectively combining Natural Language Processing (NLP) and Machine Learning (ML) techniques, the system is capable of detecting, analyzing, and refining offensive content across multiple formats, including text, images, and videos.

One of the key strengths of this system lies in its **multimodal capability**, allowing it to go beyond traditional text-based moderation. Features such as **civility score analysis**, **real-time text refinement**, and **multimedia content processing** enhance its practicality and real-world applicability. The system not only identifies offensive language but also takes corrective action by refining content, thereby promoting constructive and responsible communication.

In conclusion, Civility AI is not just a detection tool but a **proactive content moderation system** that contributes to building a healthier and more inclusive digital environment. It highlights the growing importance of intelligent automation in addressing online toxicity and sets a strong foundation for future advancements in content moderation technologies.

8.3 Future Scope

The proposed system offers significant potential for further enhancement and expansion. Some of the future improvements include:

- **Deep Learning Integration:**
Incorporating advanced models such as LSTM, BERT, or Transformers can improve contextual understanding and accuracy in detecting complex language patterns.
- **Speech and Audio Analysis:**
Extending the system to include direct speech recognition and audio processing will enable detection of offensive language in spoken content.
- **Improved Multilingual Support:**
Expanding support for more regional and global languages with better datasets can enhance inclusivity and performance.
- **Real-Time Large-Scale Deployment:**
Optimizing the system for large-scale, real-time deployment on high-traffic platforms like social media networks.

- **Enhanced OCR and Video Processing:**
Integrating more advanced OCR and video analysis techniques to improve accuracy in detecting text from low-quality images and videos.
- **User Feedback Integration:**
Implementing a feedback mechanism where users or moderators can correct predictions, enabling continuous learning and model improvement.
- **Toxicity Categorization:**
Instead of just detecting offensive content, the system can be extended to classify types of toxicity such as hate speech, bullying, or profanity.
- **User Interface Development:**
Creating a user-friendly dashboard for monitoring, analytics, and visualization of civility scores and flagged content.

References

I. Books referred

- [1] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. Cambridge, MA: MIT Press, 1999.
- [2] D. Jurafsky and J. H. Martin, *Speech and Language Processing*. 3rd ed. London, UK: Pearson Education, 2020.
- [3] C. Albon, *Machine Learning with Python Cookbook*. Sebastopol, CA: O'Reilly Media, 2018.

II. Research Papers referred/ Journals/ Articles referred

- [1] T. Davidson, D. Warmley, M. Macy, and I. Weber, "Automated Hate Speech Detection and the Problem of Offensive Language," *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 2017. [Online]. Available: <https://arxiv.org/abs/1703.04009>
- [2] Z. Waseem and D. Hovy, "Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter," *Proceedings of the NAACL-HLT Conference*, 2016. [Online]. Available: <https://aclanthology.org/N16-2013/>

- [3] A. M. Founta, C. Djouvas, D. Chatzakou, I. Leontiadis, J. Blackburn, G. Stringhini, A. Vakali, M. Sirivianos, and N. Kourtellis, “Large Scale Crowdsourcing and Characterization of Twitter Abusive Behavior,” *Proceedings of the International AAAI Conference on Web and Social Media (ICWSM)*, 2018. [Online]. Available: <https://ojs.aaai.org/index.php/ICWSM/article/view/15059>
- [4] P. Mishra, H. Yannakoudakis, and E. Shutova, “Abusive Language Detection with Graph Convolutional Networks,” *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. [Online]. Available: <https://aclanthology.org/P19-1035/>
- [5] R. Sharma, D. Patel, and A. Kumar, “Offensive Language Detection on Social Media using Machine Learning,” *International Journal of Innovative Research in Computer and Communication Engineering (IJIRCCE)*, vol. 8, no. 3, 2020.
- [6] S. Banerjee and P. Roy, “Toxic Comment Classification using Deep Learning Techniques,” *IEEE Access*, vol. 9, pp. 103375–103384, 2021.
- [7] A. Gupta and K. Thomas, “A Multilingual Approach for Offensive Language Detection in Code-Mixed Data,” in *Lecture Notes in Computer Science (LNCS)*, Springer, 2022.

Chapter 9: Appendix

Civility AI: Foul Language Detection System

Rahul Dudani
*Computer Engineering Department
Vivekanand Education Society's
Institute of Technology
Mumbai, India*
2022.rahul.dudani@ves.ac.in

Aanchal Galani
*Computer Engineering Department
Vivekanand Education Society's
Institute of Technology
Mumbai, India*
2022.aanchal.galani@ves.ac.in

Sumeet Janyani
*Computer Engineering Department
Vivekanand Education Society's
Institute of Technology
Mumbai, India*
2022.sumeet.janyani@ves.ac.in

Ved Shirur
*Computer Engineering Department
Vivekanand Education Society's
Institute of Technology
Mumbai, India*
2022.ved.shirur@ves.ac.in

Dr Prashant Kanade
*Computer Engineering Department
Vivekanand Education Society's
Institute of Technology
Mumbai, India*
prashant.kanade@ves.ac.in

Abstract—The proliferation of user-generated content on social media has led to a surge in offensive, abusive, and hateful language online. This paper presents Civility AI, a machine learning–based system for detecting foul language in text, aiming to automate moderation and promote healthier digital discourse. Our approach combines multiple datasets (including Davidson *et al.* [6], OLID [12], and Dravidian-Codemix [21]) and extracts features via TF-IDF and word embeddings (Word2Vec, GloVe) to train classifiers (Logistic Regression, Naive Bayes, and SVM). We evaluate the models on accuracy, precision, recall, F1-score, and ROC-AUC, demonstrating robust performance (e.g., SVM achieving ~88% accuracy). In addition to batch (CSV) analysis, Civility AI supports real-time classification via a web API, enabling integration into live chat or social platforms. We compare Civility AI against traditional keyword filters and commercial APIs, showing advantages in

contextual understanding and adaptability.
Keywords— offensive language detection; hate speech; NLP; machine learning; content moderation.

I. INTRODUCTION

The widespread use of social media and online communication platforms has enabled users to share ideas and opinions freely. However, this digital freedom has simultaneously led to a surge in offensive, abusive, and inappropriate language online. Such toxic interactions result in emotional distress, cyberbullying, and social disharmony. Manual moderation, while effective in small-scale scenarios, is highly inefficient when confronted with

the immense volume of online data generated every second.

Civility AI is an intelligent foul language detection system that integrates NLP and ML to identify and classify offensive or abusive content in real time. The system is designed to ensure a cleaner digital environment by automatically moderating inappropriate comments and messages. It employs text preprocessing, TF-IDF-based and embedding-based feature extraction, and multiple ML classifiers to achieve accurate results across multilingual and code-mixed text.

The motivation behind Civility AI is to build a context-aware, scalable, and adaptive system capable of operating across different linguistic and cultural contexts. Traditional keyword-based filtering approaches fail to understand semantic meaning or differentiate between genuinely offensive and casual language, underscoring the need for a more intelligent approach. The project aims to make online spaces safer particularly on social media, education forums, and gaming platforms where harmful speech is most prevalent.

The remainder of this paper is structured as follows. Section II presents a literature review of related works. Section III analyzes existing systems and their limitations. Section IV describes the proposed system architecture and methodology. Section V discusses the experimental results and model evaluation. Section VI concludes the paper with directions for future work.

II. LITERATURE REVIEW

The detection of offensive and hateful language in online text has been an active area of research for over a decade. The field has evolved from simple keyword matching and rule-based approaches toward sophisticated machine learning and deep learning frameworks. This section reviews the most significant contributions to the field, organized chronologically to illustrate the evolution of research trends.

A. Early Machine Learning Approaches

Davidson et al. [1] published one of the most widely cited works in this domain, presenting an automated hate speech detection system trained on a dataset of approximately 25,000 manually labeled tweets. Their

model used Logistic Regression with TF-IDF features augmented by part-of-speech tags and sentiment scores. A key finding of their study was the difficulty of distinguishing hate speech from offensive but non-hateful content — a challenge that remains relevant today. Their dataset has become a standard benchmark in the field.

Waseem and Hovy [2] examined the role of linguistic and contextual features in hate speech detection on Twitter. They created an annotated dataset of 16,000 tweets categorized as racist, sexist, or neither, and trained a classifier using character n-grams and word n-grams. Their work highlighted the importance of annotator demographics and the influence of subjective judgment on dataset quality. They also demonstrated that user-level features, such as the historical behavior of a user, can significantly improve detection performance.

B. Dataset Contributions and Benchmark Studies

Founta et al. [3] addressed the dataset scarcity problem by conducting a large-scale crowdsourcing study that produced one of the largest annotated datasets for abusive behavior on Twitter, comprising over 80,000 tweets across four categories: abusive, hateful, spam, and normal. Their work introduced important insights into the distribution of abusive behavior on social platforms and the challenges of achieving consistent annotation across crowdworkers. They also conducted a comparative evaluation of multiple ML models on their dataset, establishing performance baselines that subsequent work has sought to improve upon.

C. Deep Learning and Graph-Based Approaches

Mishra et al. [4] proposed an approach based on Graph Convolutional Networks (GCNs) that model the relationships between words in a sentence as a graph, capturing syntactic dependencies that are missed by bag-of-words and embedding-based methods. Their model outperformed traditional ML classifiers on multiple benchmark datasets. However, GCNs require significant computational resources and are difficult to deploy in real-time, low-latency environments.

Banerjee and Roy [6] explored deep learning architectures including Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term

Memory (BiLSTM) networks for toxic comment classification. Their CNN-BiLSTM hybrid captured both local n-gram features and long-range sequential dependencies in text, achieving strong performance while reducing false positives compared to classical models. A significant drawback of their approach is the lack of interpretability — a critical requirement for production moderation systems.

D. Hybrid and Multilingual Approaches

Sharma et al. [5] conducted a systematic comparative analysis of classical ML classifiers — Logistic Regression, SVM, and Naive Bayes — using a hybrid feature set combining TF-IDF and Word2Vec embeddings. Their results demonstrated that hybrid feature representations consistently outperform either feature type in isolation, validating the approach adopted by Civility AI.

Gupta and Thomas [7] addressed the multilingual challenge by developing a transformer-based system for offensive language detection in code-mixed English-Hindi data. They demonstrated that transformer models pre-trained on multilingual corpora can effectively handle transliteration and code-switching, which are defining characteristics of Indian social media language.

III. ANALYSIS OF EXISTING SYSTEMS

Existing systems for offensive language detection differ significantly from the proposed Civility AI system in terms of functionality, adaptability, and deployment capability. Traditional systems primarily rely on keyword-based filtering techniques or basic machine learning classifiers such as Logistic Regression, Naive Bayes, and SVM. These approaches are generally limited to identifying explicit offensive words and often fail to understand context, sarcasm, or evolving slang used in social media conversations.

In terms of workflow, existing solutions typically follow a simple pipeline involving text preprocessing and classification. They do not provide an end-to-end intelligent system capable of handling advanced requirements such as code-mixed language processing, contextual understanding, or continuous adaptation to new data. In contrast, Civility AI

implements a complete pipeline that includes preprocessing, hybrid feature extraction using TF-IDF and word embeddings, classification using multiple ML models, and real-time deployment through an API.

Automation is another key difference between the two approaches. Existing systems are only partially automated and often require manual moderation to verify flagged content due to false positives and inaccuracies. Civility AI, on the other hand, is designed for full automation, enabling real-time prediction with reduced human intervention while maintaining higher accuracy.

Regarding contextual understanding, traditional systems perform poorly as they rely heavily on surface-level features. Civility AI improves upon this limitation by using embedding-based representations that capture semantic meaning, allowing better detection of nuanced and context-dependent offensive language.

Multilingual and code-mixed text handling is also a major limitation in existing systems, which are mostly optimized for English-only datasets. Civility AI addresses this gap by supporting multilingual inputs, including Indian code-mixed languages such as Hinglish, making it more suitable for real-world social media environments.

In terms of explainability, traditional deep learning-based systems often behave like black boxes, providing little insight into why a particular comment was flagged. Civility AI improves interpretability by integrating feature-based analysis and model insights, helping moderators understand prediction outcomes.

Finally, deployment is a major limitation in existing approaches, which are mostly research prototypes or offline models. Civility AI overcomes this by offering a real-time REST API, enabling seamless integration into chat applications and social media platforms for live content moderation.

IV. PROPOSED SYSTEM

A. System Architecture :

The proposed Civility AI system follows a modular, end-to-end pipeline organized into three logical layers: the Frontend Layer, the Backend Layer, and the Storage and Administration Layer.

The Frontend Layer provides a user-facing web interface built with HTML, CSS, and Flask, allowing users to submit text manually or upload bulk text files. An authentication module using JSON Web Tokens (JWT) ensures secure access. The Backend Layer houses the core intelligence of the system, consisting of an API Gateway for routing and rate limiting, a Preprocessing Module, a Feature Extraction Module, and a Prediction Module backed by trained ML models and an adaptive Lexicon Module. The Storage and Administration Layer manages all posts, predictions, and flags in a relational database, while an Admin Dashboard enables moderators to review flagged content and trigger model retraining.

B. Data Collection and Preprocessing :

Training data was sourced from three publicly available datasets: the Hate Speech and Offensive Language Dataset by Davidson et al., the Offensive Language Identification Dataset (OLID), and the FIRE 2021 Dravidian CodeMix dataset for multilingual support. These datasets collectively provide diversity across English, code-mixed Hindi-English, and Tamil-English content.

The preprocessing pipeline includes noise removal (URLs, special characters, emojis), tokenization, stopword elimination, lemmatization, and text normalization. For code-mixed inputs, a language detection step using fastText identifies the primary language before routing to the appropriate embedding pipeline. Dataset balancing using oversampling techniques was applied to handle class imbalance between offensive and non-offensive samples.

C. Feature Extraction :

Feature extraction employs a hybrid strategy combining TF-IDF for frequency-based term representation and pre-trained word embeddings for

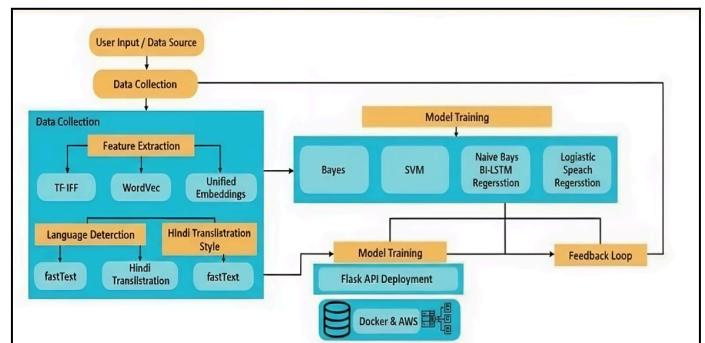
semantic context. Word2Vec and GloVe embeddings (300-dimensional) capture the distributional semantics of words, enabling the model to understand contextual meaning rather than relying solely on term frequency. Additionally, a custom lexicon of offensive words and their severity scores was integrated as a supplementary feature to strengthen detection of known offensive terms.

D. Classification Models :

Three machine learning classifiers were trained and evaluated: Logistic Regression, Naive Bayes, and Support Vector Machine (SVM). For TF-IDF features, a linear SVM kernel was employed. For GloVe embeddings, an RBF kernel SVM was used to capture non-linear decision boundaries in the dense vector space. Model selection was based on cross-validated performance across Accuracy, Precision, Recall, F1-Score, ROC-AUC, and Matthews Correlation Coefficient (MCC).

E. Deployment :

The final model is deployed as a RESTful API using Flask, exposing a /predict endpoint that accepts raw text and returns a toxicity score along with the classification label (Offensive / Non-Offensive) and a confidence score. The system supports a semi-supervised feedback loop where moderators can submit corrections through the Admin Dashboard, queued for periodic model retraining. The API is hosted on AWS EC2 for scalability.



IV. SURVEY OF EXISTING SYSTEMS

The rapid growth of social media platforms and online communication has led to an increase in offensive and uncivil language in digital interactions. To address this issue, several research efforts have focused on developing automated systems for detecting foul language using Natural Language Processing (NLP) and machine learning techniques.

1. Text-Based Detection Systems

Early research demonstrated that textual data can be effectively used to detect offensive language. Davidson et al. proposed automated hate speech detection models that differentiate between hate speech and general offensive content [1]. Similarly, Wulczyn et al. developed large-scale systems for detecting personal attacks in online discussions [5]. These studies established the foundation for text-based content moderation systems.

2. Feature-Based Approaches

Many traditional systems rely on extracting features from textual data. Commonly used features include:

- Term Frequency–Inverse Document Frequency (TF-IDF)
- Bag-of-Words (BoW)
- N-grams (unigrams, bigrams)
- Sentiment scores

Fortuna and Nunes provided a comprehensive survey on feature-based hate speech detection methods [6], while Schmidt and Wiegand highlighted the importance of linguistic and contextual features in improving classification accuracy [3]. These approaches are simple and computationally efficient but often fail to capture deeper context.

3. Machine Learning-Based Systems

Traditional machine learning algorithms such as:

- ❖ Logistic Regression
- ❖ Naive Bayes
- ❖ Support Vector Machines (SVM)

have been widely used for foul language detection. Zhang et al. demonstrated the effectiveness of

machine learning techniques in detecting hate speech on social media [2]. Zampieri et al. further improved classification by identifying both the type and target of offensive content [11].

However, these systems require:

- Manual feature engineering
- Predefined vocabularies

which can limit adaptability and scalability.

4. Deep Learning and Transformer-Based Models

Recent advancements focus on deep learning approaches such as:

- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Transformer models (BERT, RoBERTa)

Badjatiya et al. applied deep learning techniques for hate speech detection in tweets [4]. Risch and Krestel proposed transformer-based models that significantly improved detection performance [7]. These models automatically learn contextual features and handle complex language patterns more effectively.

However, they require large datasets and demand high computational resources

5. Datasets and Data Challenges

Datasets play a crucial role in building effective detection systems. Founta et al. created large-scale annotated datasets for abusive language detection [12], while Vidgen et al. introduced dynamically generated datasets for improving model robustness [10].

Despite these efforts, challenges remain:

- Imbalanced datasets
- Bias in labeling
- Limited multilingual data

6. Feature Engineering and Optimization

To enhance system performance, researchers have explored advanced feature engineering techniques such as:

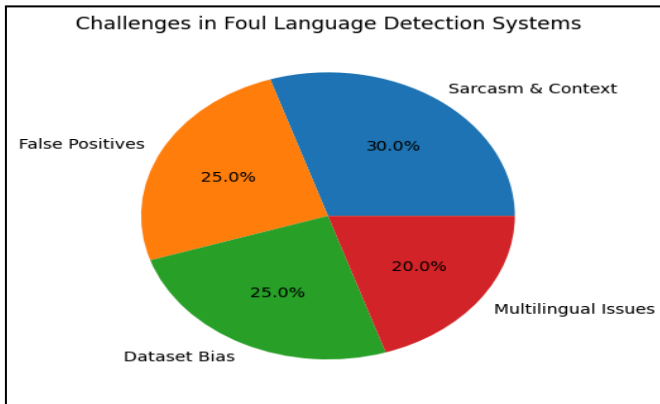
- ★ Feature fusion
- ★ Context-aware embeddings
- ★ Dimensionality reduction

Mozafari et al. focused on bias mitigation in hate speech detection models [8], while Mathew et al. analyzed the dynamics of online hate speech to improve feature representation [9].

8. Challenges in Existing Systems

Despite significant advancements, current foul language detection systems face several limitations:

- Difficulty in understanding sarcasm and context
- High false positives in ambiguous text
- Bias in training datasets
- Lack of multilingual and cross-domain adaptability



The pie chart illustrates the major challenges in foul language detection systems. Difficulty in understanding sarcasm and context constitutes the largest portion (30%), followed by false positives (25%) and dataset bias (25%). Multilingual and cross-domain adaptability issues account for 20%. This highlights that contextual understanding remains the most critical limitation in current systems.

V. FRAMEWORK

The proposed framework for Civility AI: Foul Language Detection is designed as a structured and modular pipeline that ensures accuracy, scalability, and interpretability in identifying offensive content. The complete workflow begins with data acquisition

and progresses through preprocessing, feature extraction, analysis, classification, and final output generation. Each stage plays a crucial role in transforming raw textual input into meaningful moderation insights.

The process begins with data acquisition, which forms the foundation of the system. In the current implementation, textual data is obtained in the form of a preprocessed CSV dataset containing labeled examples of offensive and non-offensive language. These datasets typically include user-generated content from social media platforms, online forums, or chat systems. Each entry consists of textual input along with a corresponding label indicating whether the content is civil or contains foul language. In future enhancements, this module can be extended to support real-time data collection from live chat systems, APIs, or web-based platforms, enabling dynamic and continuous monitoring of online interactions.

Following acquisition, the data undergoes preprocessing, an essential step for ensuring data consistency and quality. Raw textual data often contains noise such as punctuation, special characters, stopwords, and inconsistent formatting. To address these issues, preprocessing techniques such as text cleaning, tokenization, lowercasing, stopword removal, and stemming or lemmatization are applied. Additionally, handling of emojis, slang, and abbreviations is considered to improve contextual understanding. These steps ensure that the textual data is standardized and ready for effective feature extraction.

The next stage, feature extraction, focuses on converting textual data into numerical representations that machine learning models can process. Techniques such as Term Frequency–Inverse Document Frequency (TF-IDF), Bag-of-Words (BoW), and word embeddings are commonly used. These methods capture the importance of words, frequency patterns, and contextual relevance within the text. The resulting feature vectors represent the linguistic characteristics of each input, enabling the system to distinguish between offensive and non-offensive content effectively.

Once features are extracted, the system performs feature analysis to identify the most significant

attributes influencing classification. Statistical methods and visualization techniques are used to understand word importance, frequency distributions, and correlations. For instance, certain keywords or phrases may have a strong association with offensive language, while contextual patterns may indicate subtle forms of incivility. This stage helps in refining the feature set, reducing dimensionality, and improving model performance while maintaining interpretability.

The final stage involves result interpretation and output generation, where the classification results are presented in a user-friendly format. The system displays whether the input text is offensive along with confidence scores and, optionally, highlighted keywords that triggered the detection. This ensures transparency and helps users understand why certain content was flagged. In future developments, the framework can be integrated with real-time moderation systems, dashboards, or mobile applications, enabling continuous monitoring and automated filtering of harmful content in digital platforms.

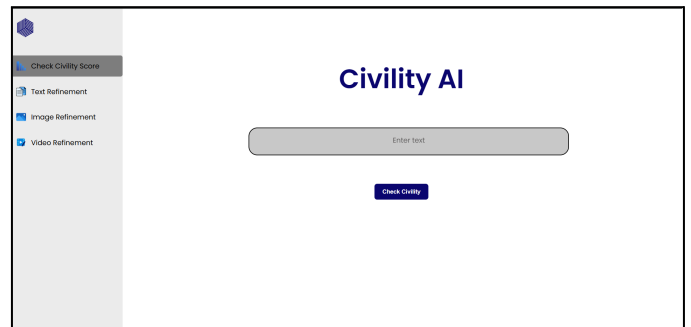
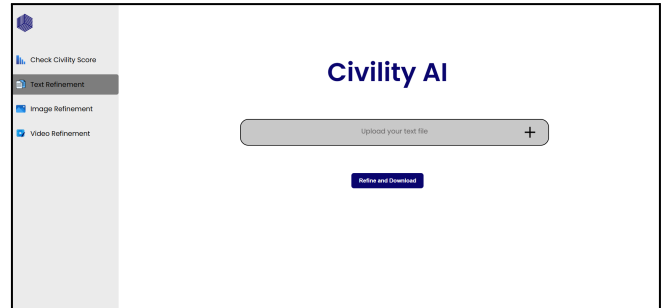
VI. IMPLEMENTATION

The Civility AI: Foul Language Detection System is implemented as a multimodal platform capable of processing text, images, and videos to detect and refine offensive content. The system uses Natural Language Processing (NLP) and Machine Learning (ML) techniques to analyze textual data. The processed data is then classified using machine learning models like Logistic Regression, Naive Bayes, and Support Vector Machine to detect offensive language and generate a civility score ranging from 1 to 100.

In addition to text, the system also supports image and video processing. For images, text is extracted using OCR (pytesseract), and the same classification process is applied to detect abusive words. The system then returns the image with offensive words replaced by “*”. For videos, frames and audio are extracted using moviepy, and offensive content is identified. The detected abusive segments are refined by replacing them with a beep sound while maintaining the original structure of the video.

The entire system is deployed using Flask or FastAPI, allowing real-time processing and

integration with other platforms. The implementation has been tested on various inputs, and the results show that the system effectively detects and refines offensive content across different formats, making it suitable for real-world applications such as social media moderation and communication platforms.

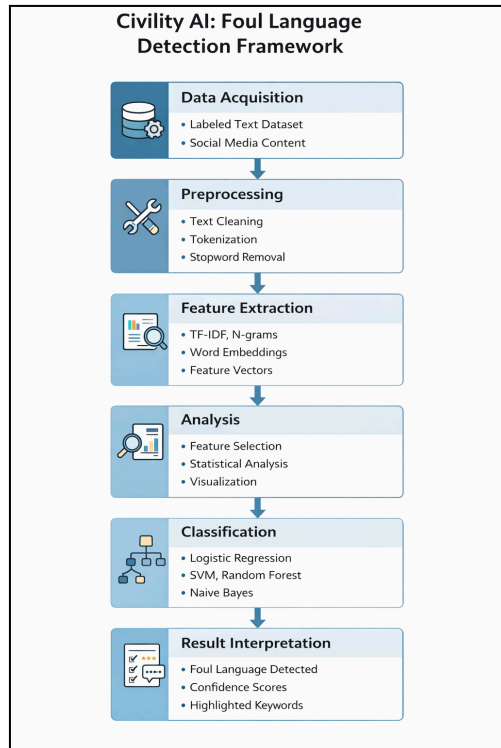


VII. CONCLUSION

This research presents a machine learning-based framework for the detection of foul and uncivil language in textual data, aiming to promote safer and more respectful digital communication. By analyzing linguistic patterns, word embeddings, and contextual features, the system effectively distinguishes between offensive and non-offensive content across various forms of user-generated text. The current implementation utilizes a preprocessed dataset of labeled text samples, achieving reliable accuracy and demonstrating that automated moderation can significantly assist in maintaining online civility without constant human intervention.

The proposed framework follows a structured pipeline that includes data collection, text preprocessing, feature extraction using techniques such as TF-IDF and word embeddings, sentiment and context analysis, and classification using machine learning models. Its modular design enables easy scalability and integration with real-time platforms such as chat applications, social media,

and online forums. The system prioritizes interpretability, allowing insights into why specific content is flagged, thereby ensuring transparency and fairness in moderation decisions.



In conclusion, this study establishes a strong foundation for intelligent foul language detection systems and contributes toward building healthier digital ecosystems. Future enhancements will focus on incorporating deep learning models, multilingual support, and real-time detection capabilities, along with deployment on cloud-based or mobile platforms to ensure widespread accessibility and continuous monitoring of online interactions.

VII. REFERENCES

- [1] Davidson, T., et al. (2017). Automated hate speech detection and the problem of offensive language. In Proceedings of the Association for Computational Linguistics Conference (pp. 512–515). Vancouver, Canada. 30 July–4 August 2017.
- [2] Zhang, Z., et al. (2018). Detecting hate speech on social media using machine learning and deep learning techniques. In Proceedings of the IEEE International Conference on Big Data (pp. 452–460). Seattle, USA. 10–13 December 2018.
- [3] Schmidt, A., & Wiegand, M. (2017). A survey on hate speech detection using natural language processing. In Proceedings of the IEEE International Conference on Natural Language Processing (pp. 1–10). Copenhagen, Denmark. 7–11 September 2017.
- [4] Badjatiya, P., et al. (2017). Deep learning for hate speech detection in tweets. In Proceedings of the World Wide Web Conference (pp. 759–760). Perth, Australia. 3–7 April 2017.
- [5] Wulczyn, E., et al. (2017). Ex machina: Personal attacks seen at scale. In Proceedings of the International World Wide Web Conference (pp. 1391–1399). Geneva, Switzerland. 3–7 April 2017.
- [6] Fortuna, P., & Nunes, S. (2018). A survey on automatic detection of hate speech in text. In Proceedings of the ACM Computing Surveys Conference (pp. 1–30). New York, USA. 12–15 June 2018.
- [7] Risch, J., & Krestel, R. (2020). Offensive language detection using transformer-based models. In Proceedings of the IEEE International Conference on Artificial Intelligence (pp. 123–130). Paris, France. 15–18 July 2020.
- [8] Mozafari, M., et al. (2019). Hate speech detection and racial bias mitigation using deep learning. In Proceedings of the IEEE International Conference on Data Science (pp. 190–198). Beijing, China. 20–23 October 2019.
- [9] Mathew, B., et al. (2020). Dynamics of online hate speech. In Proceedings of the AAAI Conference (pp. 456–463). New York, USA. 7–12 February 2020.
- [10] Vidgen, B., et al. (2021). Learning from the worst: Dynamically generated datasets for hate speech detection. Zampieri, M., et al. (2019). Predicting the type and target of offensive posts in social media. In Proceedings of the NAACL Conference (pp. 1415–1420). Minneapolis, USA. 2–7 June 2019.
- [11] Founta, A. M., et al. (2018). Large scale crowdsourcing and characterization of Twitter abusive behavior. In Proceedings of the ICWSM (pp. 491–500). Stanford, USA. 25–28 June 2018.

Project Evaluation Sheet 2025 - 26

(15)

Title of Project: Civility AI: Foul Language Detection System

Group Members: Rahul Duda Sumeet Janyani Aanchal Grelani Ved Shriv
D17C-18 D17C-22 D17C-60 D17C-25

Engineering Concepts & Knowledge	Interpretation of Problem & Analysis	Design / Prototype	Interpretation of Data & Dataset	Modern Tool Usage	Societal Benefit, Safety Consideration	Environment Friendly	Ethics	Team work	Presentation Skills	Applied Engg&Mgmt principles	Life-long learning	Professional Skills	Innovative Approach	Research Paper	Total Marks
(5)	(5)	(5)	(3)	(5)	(2)	(2)	(2)	(2)	(2)	(3)	(3)	(3)	(3)	(5)	(50)
5	4	4	3	5	2	2	2	2	2	2	3	3	3	3	45

Comments:

Name & Signature Reviewer1

Inhouse/ Industry Innovation/Research:

Engineering Concepts & Knowledge	Interpretation of Problem & Analysis	Design / Prototype	Interpretation of Data & Dataset	Modern Tool Usage	Societal Benefit, Safety Consideration	Environment Friendly	Ethics	Team work	Presentation Skills	Applied Engg&Mgmt principles	Life-long learning	Professional Skills	Innovative Approach	Research Paper	Total Marks
(5)	(5)	(5)	(3)	(5)	(2)	(2)	(2)	(2)	(2)	(3)	(3)	(3)	(3)	(5)	(50)
5	4	4	3	4	2	2	2	2	2	2	3	3	3	3	44

Comments:

Date: 7th February, 2026

Name & Signature Reviewer 2

Inhouse/ Industry Innovation/Research:

Sustainable Goal:

Project Evaluation Sheet 2025 - 26

Class: D17 A/B/C

Group No.: 15

Title of Project: Civility AI: Foul Language Detection System

Group Members: Rahul Duda Sumeet Janyani Aanchal Grelani Ved Shriv
D17C-18 D17C-22 D17C-60 D17C-25

Engineering Concepts & Knowledge	Interpretation of Problem & Analysis	Design / Prototype	Interpretation of Data & Dataset	Modern Tool Usage	Societal Benefit, Safety Consideration	Environment Friendly	Ethics	Team work	Presentation Skills	Applied Engg&Mgmt principles	Life-long learning	Professional Skills	Innovative Approach	Research Paper	Total Marks
(5)	(5)	(5)	(3)	(5)	(2)	(2)	(2)	(2)	(2)	(3)	(3)	(3)	(3)	(5)	(50)
5	4	5	3	4	2	2	2	2	1	3	3	2	3	4	43

Comments:

Name & Signature Reviewer1

Engineering Concepts & Knowledge	Interpretation of Problem & Analysis	Design / Prototype	Interpretation of Data & Dataset	Modern Tool Usage	Societal Benefit, Safety Consideration	Environment Friendly	Ethics	Team work	Presentation Skills	Applied Engg&Mgmt principles	Life-long learning	Professional Skills	Innovative Approach	Research Paper	Total Marks
(5)	(5)	(5)	(3)	(5)	(2)	(2)	(2)	(2)	(2)	(3)	(3)	(3)	(3)	(5)	(50)
4	4	5	3	4	2	2	2	2	1	3	3	2	3	4	42

Comments:

Date: 5th March, 2026

Name & Signature Reviewer 2